國立臺灣科技大學

工業管理系

碩士學位論文

學號：M10901106

智動化揀貨系統之揀貨流程最佳化研究

Optimization of Order Picking Process in Robotic Mobile Fulfillment System

研 究 生：吳沛澄

指導教授：周碩彥 博士

郭伯勳 博士

中華民國一百一拾一年八月

M10901106

碩士學位論文指導教授推薦書

Master's Thesis Recommendation Form

系所： Department/Graduate Institute	工業管理系 Department of Industrial Management
姓名： Name	吳沛澄 WU, PEI-CHENG
論文題目： (Thesis Title)	智動化揀貨系統之揀貨流程 最佳化研究 Optimization of Order Picking Process in Robotic Mobile Fulfillment System

係由本人指導撰述，同意提付審查。

This is to certify that the thesis submitted by the student named above, has been written under my supervision. I hereby approve this thesis to be applied for examination.

指導教授簽章：
Advisor's Signature

共同指導教授簽章（如有）：
Co-advisor's Signature (if any)

日期：
Date(yyyy/mm/dd)

111 / 8 / 23

M10901106

碩士學位考試委員審定書

Qualification Form by Master's Degree Examination Committee

系所： Department/Graduate Institute	工業管理系 Department of Industrial Management
姓名： Name	吳沛澄 WU, PEI-CHENG
論文題目： (Thesis Title)	智動化揀貨系統之揀貨流程 最佳化研究 Optimization of Order Picking Process in Robotic Mobile Fulfillment System

經本委員會審定通過，特此證明。

This is to certify that the thesis submitted by the student named above, is qualified and approved by the Examination Committee.

學位考試委員會

Degree Examination Committee

委員簽章：
Member's Signatures

郭伯勳

周碩彥

陳彥旭

指導教授簽章：
Advisor's Signature

周碩彥

共同指導教授簽章（如有）：
Co-advisor's Signature (if any)

郭伯勳

系所（學程）主任（所長）簽章：
Department/Study Program/Graduate Institute Chair's Signature

郭伯勳

日期：
Date(yyyy/mm/dd)

111 / 8 / 23

ABSTRACT

Robotic Mobile Fulfillment System (RMFS) is a parts-to-picker system well suited to the e-commerce business. However, there are many aspects to improve the system. One of the most critical issues is the order picking process. The decision problems of the order picking process in RMFS can be divided into Pick Order Assignment (POA) and Pick Pod Selection (PPS). Most papers solve the static version of the problem. To execute order picking efficiently in a different situation, this research proposes a heuristic method to solve the problemon a real-time scale. The proposed method dynamically optimizes the order batching size, corresponding to the current situation. Compared to most literature, instead of minimizing the pod visit, the objective function of our proposed model is maximizing the pile-on, which is an essential indicator of achieving high throughput. The maximization of pile-on for every batch can increase the efficiency of order picking and decrease the system cost. Two types of pile-on are examined. One is basic – the number of SKUs provided by each pod visit. The other one is advanced: it additionally considers completed orders. A heuristic applying the Greedy Randomized Adaptive Search Procedure (GRASP) is proposed to solve the problem efficiently and effectively. The proposed order picking model is implemented in the agent-based simulation system to test its effectiveness. Several indicators are used to test the performance of the proposed method compared to the benchmark. The results show that the proposed method significantly outperforms the benchmark in most indicators. Moreover, a new indicator is shown that strongly correlates with throughput order.

Keywords: Robotic Mobile Fulfillment System (RMFS), Order picking, Metaheuristic, Local search, Simulation

摘要

智動化揀貨系統（Robotic Mobile Fulfillment System, RMFS）是眾所周知且常應用於電子商務業務中的訂單揀貨系統。但是，該系統還有很多方面需要改進。最關鍵的問題之一是關於訂單揀貨流程。 RMFS中訂單揀貨流程的決策問題可以分為兩個：揀貨訂單指派（POA）和揀貨貨架選擇（PPS）。大多數文獻解決了該問題的靜態版本。為了在不同的情況下有效地執行訂單揀選，本研究提出了一種啟發式方法來解決即時的問題。所提出的方法根據當前情況動態優化訂單批量大小。與大多數文獻最小化貨架訪問相比，所提出模型的目標函數是最大化實現高吞吐量的重要指標: Pile-on。最大化每批次的Pile-on可以提高揀貨效率，降低系統成本。本研究探討了兩種類型的Pile-on。一個是常用於許多文獻的Pile-on——每次貨架訪問揀貨站可提供的SKU 數量。另一種是額外考慮完成訂單數量的版本。為了有效地解決問題，本研究提出了一種應用貪婪隨機自適應搜索過程（Greedy Randomized Adpative Search Procedure, GRASP）元啟發式的啟發式算法。本研究將所提出的訂單揀選模型實現於代理人模擬系統，並使用了數個指標來測試所提出方法的性能以測試其有效性。結果表明，所提出的方法在大多數指標上都顯著優於基準方法，除此之外，本研究發現一項新的指標與訂單吞吐量有很強的相關性。

關鍵字: 智動化揀貨系統（Robotic Mobile Fulfillment System, RMFS）、訂單揀貨系統、元啟發式演算法、區域搜尋、模擬

ACKNOWLEDGEMENT

Firstly, I would like to express my sincerest gratitude to my advisor, Prof. Shuo-Yan Chou, who has supported and guided me throughout my research and thesis. His ideas, kindness, advice, and passion always inspire and motivate me to enhance my work and achieve a great outcome. I would also like to acknowledge Prof. Po-Hsun Kuo as my co-advisor and Prof. Jen-Ming Chen as my thesis defense committee for their encouragement, insightful comments, evaluation, and suggestions for my research.

Secondly, I would also like to give my appreciation to all my labmates/friends in the Center of IoT Innovation (CITI), especially Kiva teammates: Moritz, Agnes, Chaterine, Edwin, Tina, Rasyid, Dennis, Ben, Ian, David, and all who involves, for their friendliness, kindness, and support during my work in this project these past two years. And I also want to give immense gratitude to others who provide me with lots of help, patience, guidance, care, and support: Indie, Rafi, Ryanda, Joe, Phoebe, Molly, Kevin, and all other members. And I would also like to thank my friends who always support, love, and encourage me, especially my lovely friend 9mbb. Furthermore, I must express my profound gratitude to my parents and siblings for providing me with unfailing support and continuous encouragement throughout my years of study and through the process of researching and writing this thesis. This accomplishment would not have been possible without them. Thank you.

Andy Wu(吳沛澄)

Taipei, August 2022

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

CHAPTER 1
INTRODUCTION

The current pandemic situation has led people to purchase more goods online. As a result, the profit of E-commerce is estimated to grow by 8.1% from 2020 to 2024 (Statista, 2020). On the other hand, the delivery schedule of E-commerce is tight compared to standard orders from 2-day delivery, next-day delivery, or even same-day delivery (Absolunet, 2020). So, handling lots of volatile, variant items is a big problem that bothers e-commerce merchants. For example, an E-commerce platform, Amazon, provides up to 372 million products online for sales (Zhou et al., 2018). Therefore, selecting an Order Picking System (OPS) suitable for handling this kind of order is necessary. That is because 55% of operational warehousing costs are caused by order picking (De Koster et al., 2007) (see Figure 1-1).

Order picking systems (OPS) can be categorized into five branches: picker-to-parts, parts-to-picker, pick-to-box, pick-and-sort, and utterly automated picking(Dallari et al., 2009). The classification is shown in Figure 1-2. Each OPS has its pros and cons. The traditional OPS, parts-to-picker, can not handle the E-commerce order efficiently. It is because E-commerce orders have lots of SKU (Boysen et al., 2017). In the traditional picker-to-parts system, the picker walks through the warehouse to pick SKU. It is inefficient to let pickers walk long distances and pick SKUs of a large assortment. Compared to the traditional one, the Robotic Mobile Fulfillment System (RMFS) was invented to process this kind of order. RMFS is an advanced parts-to-picker warehouse system. The advantages of RMFS include decreasing picker moving distance, simplifying human work, and its flexibility and scalability (MWPVL, 2012). It is reported that picker productivity is five to six times higher than manual picking (Wulfraat, 2012). The system has been used in many E-commerce and logistic companies, such as Amazon (Merschformann et al., 2018) and Ally Logistic Property.

In the RMFS system, the AGV delivers the pod from the storage area to the picking stations. At the picking station, the picker stays in the picking station to take the SKU from pods and put it in bins. The bins are placed on a shelf in the picking station area to collect the SKUs of an order. After the SKUs of the current order in

the bin are collected entirely, the picker presses a button, releasing the bin for the following order. However, suppose the number of orders is large, and the SKU per order has many variances. In that case, the order picking process must be optimized correctly to lower the throughput time.

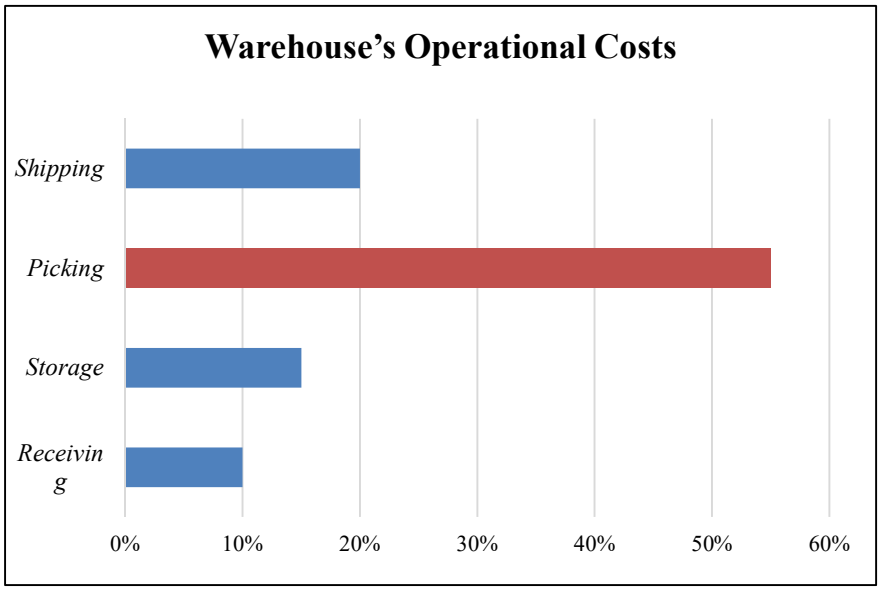

Figure 1-1 Portion of warehouse's operational costs

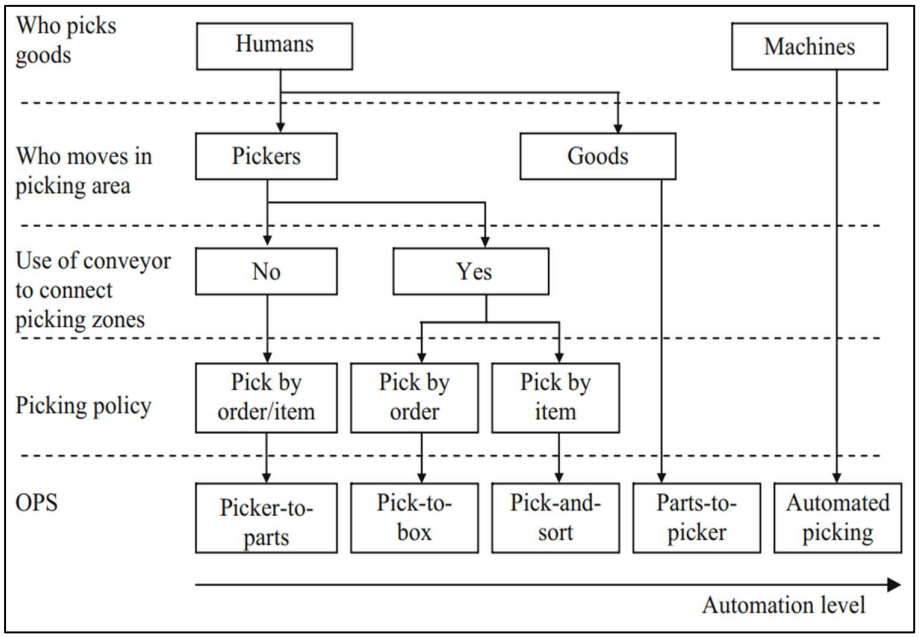

Figure 1-2 Classification of OPS

This research focuses on how to satisfy the customer demand so that the cost issue is ignored. This way, throughput becomes one of the most critical performance measurements in RMFS. There are many ways and aspects to achieve high throughput. One of the most vital operations is the order picking process because it directly influences how the customer's demand can be satisfied in a short time or same-day delivery (Lamballais et al., 2017). The order picking process includes Pick Order Assignment (POA) and Pick Pod Selection (PPS) (Merschformann et al., 2019).

POA represents the assignment of orders to picking stations and the sequence of orders occupying bins in picking stations. The number of bins is limited so that only bin numbers of orders can be processed parallelly. The optimized order sequence results in many SKUs that can be picked when a pod visits. PPS represents pods' assignment to pick stations and the sequence of pods visiting picking stations. Both influence each other intensely and should be considered together. One of the correlated indicators of throughput determined by POA and PPS is pile-on value, which traditionally indicates how many units can be picked in a pod visiting a picking station. The high pile-on value positively correlates to high throughput (Merschformann et al., 2019).

Merschformann et al. (Merschformann et al., 2019) implements RMFS in RAWSim-O (Merschformann et al., 2017), which is a discrete event simulation platform for RMFS. The author proposes the policy-based rule to decide on POA and PPS. The start condition of solving POA is that an available bin appears. The structure of its solution is to find an order to assign it to the bin. Moreover, the start condition of solving PPS is that an AGV becomes available. The structure of its solution is to find a pod to deliver. The four ways to improve the processes are stated below:

(1) Considering the orders in the bins and order pool simultaneously is essential in order picking. Processing the orders in the bins increases throughput immediately. Moreover, examining the orders in the order pool leads to better assignment and scheduling for newly available bins. Therefore, solving POA by synchronizing the two aspects is one of the objectives of this research.

(2) The POA and PPS are decided separately. However, the overall planning

generates a better result.

(3) Both POA and PPS provide one order and pod in each decision. Therefore, it is intuitively that deciding on several orders and pods yields a better result with the concept of order batching. However, it is unclear to know the batch size either for POA or PPS. As a result, this research aims to find the best batch size. The best batch size indicates the average pile-on of scheduled pods being maximized. That means those scheduled pods are well utilized.

(4) Inherit to the previous point, the sequence of orders and pods needs to be considered to implement order batching in POA and PPS. That is because the picking station can only accommodate limited numbers of orders and pods.

To sum up the opportunities, this research answers the following questions. When we are about to make the decision on POA and PPS:

(1) Which pods to schedule?

(2) Which corresponding orders to schedule?

(3) How many pods to schedule in one batch? (*i.e.,* scheduling one, two, or more pods?)

(4) How many orders to schedule in one batch? (*i.e.,* scheduling one, two, or more orders?)

(5) What is their sequence if we schedule more than one pod and order?

The contributions of this research are:

(1) Formulating a mathematical model to find the best order and pod batches in the decision to POA and PPS.

(2) Developing a hybrid heuristic that can solve a large instance of the problem effectively and efficiently.

(3) Compared to most previous research, which minimizes pod visits to improve throughput, this research does it by maximizing pile-on value. An agent-based simulation is used to demonstrate the benefit of solving POA and PPS with maximizing pile-on.

The organization of this study is divided into five major chapters. Chapter 1 presents the background, motivation, and objective of the study. The literature review is explained in Chapter 2. Chapter 3 presents system modeling and problem description and describes the heuristic to solve the problem efficiently and

effectively. Chapter 4 presents the proposed heuristic's effectiveness, the proposed model's influence on the RMFS system, and the findings related to indicators to improve the throughput. Finally, chapter 5 presents the conclusion and future research opportunities. Figure 1-3 shows the structure of this research.

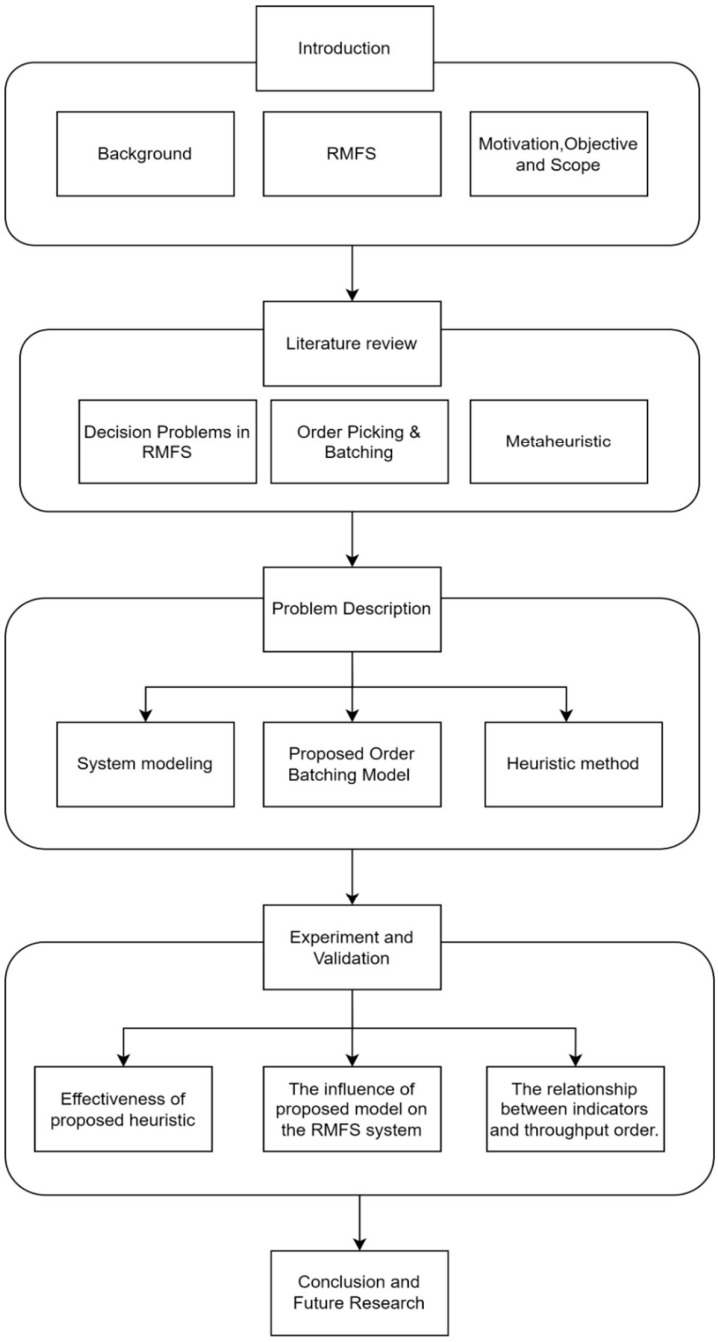

Figure 1-3 Structure of Research

CHAPTER 2
LITERATURE REVIEW

2.1 Decision problems in RMFS

The decision problems in RMFS can be categorized into three levels, from top to bottom: strategic, tactical, and operational (Merschformann et al., 2018). The strategic level represents long-term decisions that are hard to change instantly. That includes layout design, facilities, and capacity planning. Then, the tactical level represents middle-term decisions that can be made weekly or monthly. That includes resource allocation, zoning, and inventory level. Finally, the operational level represents short-term, real-time decisions that should be made and changed instantly. The operational level includes Order Assignment (OA), Pod Selection (PS), Pod Storage Assignment (PSA), Task Allocation (TA), and Path Planning (PP). The OA can be categorized into Pick Order Assignment (POA) and Replenishment Order Assignment (ROA). The PS can also be categorized into Pick Pod Selection (PPS) and Replenished Pod Selection (RPS).

Table 2-1 Classification of decision problems in RMFS

Strategic Level – Decisions at time of warehouse constrution Station Placement Storage Area Dimensioning
Tactical Level – Decisions at start workweek, workday or shirt Number of Pods per SKU Ratio between numbers of picking and replenishment stations Replenishment Level Resource Reallocation
Operational Level – Decisions in real-time Order Assignment(OA) Pod Selection (PS) Pod Storage Assignment(PSA) Task Allocation(TA) Path Planning(PP)

In strategic level problems, Aldarondo, FJ, and Y.A. Bozer (Aldarondo & Bozer, 2022) used the travel times model to estimate AGV traveling distance under different order assignment policies, picking station placement, different shapes of field, length-to-width ratio, and numbers of pods per order. They suggest the best configuration would be a square-shaped layout with picking stations on four sides. Placing the picking station on the long side is better if the layout is not square. That is because the workstations' placement on short sides makes AGV travel through

the long side.

Zou. B. et al. (Zou et al., 2017) also dealt with strategic-level problems. First, the author discussed optimizing the size of the pod batch and field ratio. They use a semi-open queuing network and a two-phased approximate method to solve the problem. The result shows the optimal width of the pod batch reduces with the width-to-length ratio.

Lamballaiset al. (Lamballais et al., 2022) dealt with the tactical-level problem in real-time. The paper uses the Markov decision process to decide how to allocate the resource when the demand is varied. The resource includes picking stations, replenishment stations, and AGV. In this literature, a station can change between the function of picking or replenishment. Also, the trade-off to process picking and replenishment order was considered. For example, if we put too many resources into picking orders, the inventory would no longer support efficient order picking.

Conversely, the opposite situation tends to increase tardiness in the picking order. Both situations take expensive costs in the fluctuated and peak demand. Finally, the author proposed four heuristics: fixed (baseline), demand-dependency, cost-dependent, and state-dependent policy. The experiment results showed the amount of peak demand and the duration of peak demand influence the selection of policies. Furthermore, the result of continually reallocating resource is better than the fixed one, and their proposed policy save lots of costs if the number of AGVs is limited.

Another important topic at the tactical level is the SKU storage problem. Yuan, R., S.C. Graves, and T. Cezik (Yuan et al., 2019) developed a fluid model to analyze the classification of SKUs and placement in the warehouse. They suggest the SKU having similar popularity should be put together in a pod. Those pods consisting of more popular SKU should be closed to the picking station. Li. et al. (Li et al., 2020) developed a step-by-step method to solve two problems: the combination of SKU in a pod and where the pods are located in the warehouse. For the first problem, they utilized the association and clustering method to analyze the relationship of SKU. SKU usually being ordered together should be put in a pod so that the number of pod visiting picking station time is reduced. In the second step,

they use a turnover-rate-based decentralized storage policy to optimize the location of pods. The concept is to disperse those popular pods in the warehouse to avoid congestion of AGVs. Their results show that the proposed policy improves energy consumption and traveling distance.

2.2 Order picking and order batching

Order picking is a series of processes to retrieve products from the warehouse inventory to fulfill customer orders. It involves the process of clustering and scheduling the customer orders, releasing them to the floor, picking the items from storage locations, and disposing of the picked items. Order batching is a necessary procedure and concept to solve assignment and scheduling problems in order picking. It combines similar orders in a batch so that they are processed efficiently. The similarities of orders are defined in a lot of ways. For instance, Ho, Y.-C., and Y.-Y. Tseng (Ho & Tseng, 2006) utilized the location characteristic. They batch the orders whose SKUs are put closely in the warehouse. (Chen & Wu, 2005) utilized the apriori algorithm to find the SKU ordered together and batch those orders together. A classical way to categorize order batching is by its dynamicity. Off-line(static) order batching studies the process of fulfilling and batching a fixed set of orders. All the information on existing orders is given, and the decision is made without considering the previous order batches or orders arriving in the future. Concerning Online (dynamic) order batching, orders arrive continually and stochastically. That means the order picking is executing while new orders keep entering the system. In this case, determining the size and interval (time window) of order batches is critical.

The order batching problem is one of the problems in the traditional picker-to-parts system. Several methods are utilized for the offline order batching problem, including policy-based heuristics, saving algorithms, or metaheuristics. Ho and Tseng (Ho & Tseng, 2006) introduce several seed selection policies and accompanying order selection policies to minimize the routing distance. Most of their policies are location or aisle based. The authors showed that not only seed order or accompanying order selection influence objective value significantly but also their combination. The saving algorithm is also popular in the order batching problem(Clarke & Wright, 1964). The concept is to combine two orders which can

save the most traveling distance together step by step until a complete solution is constructed. The metaheuristic is also a popular way to solve the offline order batching problem. Cheng, C.-Y. et al. (Cheng et al., 2015) proposed a hybrid-heuristic combining particle swarm optimization (PSO) and ant colony optimization (ACO) to solve the order batching problem. The author uses PSO to batch the orders, find the appropriate order batch sizes, and use ACO to generate the route. The fitness from ACO is used to update the parameter in PSO. The method is proved statistically significant compared to the CPLEX solver and benchmark.

Zhang, J., et al. (Zhuang et al., 2021) proposed a hybrid rule-based algorithm to solve assignment and sequence problems under a variant order environment for the online order batching problem. The author stated that fixed time windows and variant time windows perform worse in the variant order environment. The objective function of this research is to minimize the maximum completion time of all the batches. They separate the overall problem into two parts. First, group the orders into batches and assign those batches to the picker. Then, the frequency of orders arriving in a predefined time interval is recorded. They define three categories for the interval based on that. If the quantity of orders in the current interval is few, then this time interval is defined as type A (off-peak). If the quantity of orders coming in the current interval is normal, then this time interval is defined as type B (normal). If the quantity of orders coming in the current interval is very high, then this time interval is defined as type C (peak).

For the type A time interval, it is not suitable to make them a batch. Starting a new trip for picking small quantities is wasteful. Therefore, the orders in type A time interval would be assigned to the busy picker, the one currently picking the item. They use the save algorithm in this type A case to assign and schedule orders. The orders arriving in it are considered in a batch for the type B time interval. For the type C time interval, the orders arriving in it would be grouped into several batches. In this type C case, they use a seed algorithm which is considering location and arrival time. The batches generated in either type B or type C are assigned to the worker by the list processing algorithm (Graham, 1966), which solves the m-machine scheduling problem. The algorithm can not only minimize the turnover time and also balance the picker load. The result shows that the improvement is

significant compared to the other traditional online order batching methods.

Compared to the research on the picker-to-parts system, which usually minimizes the traveling distance, tardiness, or maximizes turnover time, most of the research discusses the order picking model at the operational level in RMFS, which focus on the minimization of pod visit. The definition of a pod visit is one pod visiting one station. If the pod hops to another picking station after finishing its task in the current station, then it is another pod visit. The reason for minimizing this objective proxy value is explained in (Boysen et al., 2017; Zhuang et al., 2021), which are:

1) It reduces the number of AGVs used potentially. This way, the cost of energy and assets decreases.
2) Every pod visit happens to generate pod transition costs. This is because every pod transition makes the picker idle. Once the pod visit is minimized, the idle time of the picker caused by pod transition can be reduced.
3) The pile-on value is a significant indicator to optimized throughput in RMFS (Merschformann et al., 2018). The definition of pile-on is

$$\text{pile\_on value} = \frac{Number\ of\ SKUs\ being\ picked}{total\ pod\ visit} \quad \textbf{(2-1)}$$

where the pile-on value represents how many numbers of SKUs the picker can pick from a single pod visit on average. If we fixed the numerator (it is reasonable in the static problem whose orders are known before making a decision), minimizing the denominator achieves the maximum pile-on value. On the other hand, it is hard to maximize the pile-on value inversely because the objective value would become non-linear compared to the linear one of minimizing pod visits.

In sum, minimizing pod visits is the big picture of solving the order picking model at the operational level in many research studies.

Optimizing the order and pod sequence is critical to minimizing the pod visit. In the configuration of picking stations of RMFS, there are bins for storing the picked SKU temporarily. Every bin is dedicated to an order until it is picked thoroughly. Then, the complete order would be removed so that the next order could be assigned to the bin. An order can't be picked without the occupation of a bin.

That is the reason the decision on order sequence is essential. The picker can only pick SKU from one pod at a time. So the pod sequence is also critical. A good synchronization of these two sequences achieves minimizing pod visits. Some previous research only considers the assignment of order and pod to station (Shi et al., 2021; Valle & Beasley, 2021). However, Zhuang, Y. et al. (Zhuang et al., 2021) proved that only considering the assignment couldn't achieve the minimization of pod visits. The author tests it with a two-stage method. In the first stage, the model of assignment of order and pod to station is used. After that, the model of minimization of pod visit in a single picking station case proposed by Boysen, N., D. Briskorn, and S. Emde (Boysen et al., 2017) is utilized to optimize the order and pod sequence based on the result of the assignment. The result shows that directly optimizing the order and pod sequence is better. The following paragraphs review the optimization of order and pod sequence problems.

Boysen, N., D. Briskorn, and S. Emde (Boysen et al., 2017) proposed a model to deal with the problem in a single picking station case. The author focuses on how to solve POA and PPS within a static order pool. That means, given an order pool $\boldsymbol{O}$, how to process all the orders in $\boldsymbol{O}$ with minimized pod visit. This problem is called MROP. The complexity of this problem is proved NP-hardness even if there is only one picking station. The author decomposes the problem into two subproblems: based on the order sequence to find the pod sequence with minimized pod visit (MROP-RS) and based on the pod sequence to find and feasible order sequence (MROP-OS). Unfortunately, the two subproblems are still hard to solve. The MROP-RS is proved NP-hard strongly, and the MROP-OS is proved strongly NP-complete. To solve these two subproblems efficiently and effectively, the author proposes dynamic programming methods combined with beam search because the branches are growing exponentially. Based on the theories and experiments in this paper, a general concept to solve MROP is finding a feasible order sequence and then finding a feasible pod sequence according to it. Because their experiment of finding order sequence after pod sequence has a worse result. The author applied a simulated annealing metaheuristic (Bertsimas & Tsitsiklis, 1993) as a search mechanism to find a good quality solution. Swapping two orders in the order sequence is the operator in each iteration. After the swapping, their proposed

method to find pod sequences based on order sequence is utilized. The new pod sequence is derived, and the size of it (number of pod visits) is the fitness value.

(Zhuang et al., 2021) proposed multiple picking stations version of the problem. Pod conflict is considered in their problem. They first generate a good quality initial solution with the proposed constructive heuristic in their proposed heuristic. Before introducing the way they generate initial order and pod sequence, two functions are utilized to quantify the relationship between pods and orders. They calculate the fulfillment rate of pod j to order i. The function of the fulfilling rate is shown below:

$$fr_{i,j} = \frac{\sum_{s \in S} A_{i,s} I_{j,s}}{\sum_{s \in S} A_{i,s}} \ , \forall i \in O, j \in P \quad \textbf{(2-2)}$$

The equation (2-2) shows how much pod j can fulfill order i. After that, the supply rate for each pod j is calculated:

$$sr_j = \sum_{i \in O} fr_{i,j} \quad \textbf{(2-3)}$$

The equation (2-3) presents how much pod j can fulfill orders in the order pool. They start the process of generating an initial solution by scheduling the pod with the highest supply rate to the picking station. It is because the highest supply rate pod can fulfill the most orders intuitively. The corresponding orders are scheduled consequently. The next pod is found by considering the partial orders in the bin. The pod that can process the most is chosen. Then, corresponding orders can be fulfilled by the chosen pod scheduled if there are available bins. Repeating the above procedures iteratively generates the order and pod sequences. After that, they use simulated annealing to operate the local search. Two operators are proposed, swapping two orders in the order sequence or relocating the order to a new position. Furthermore, they utilize an adaptive large neighborhood search algorithm(ALNS) to escape the local optimum.

The two studies above studied on the optimization of a given set of orders. Some authors have different experiment environments to solve different situations in the actual industry. For example, Merschformann (Merschformann et al., 2019) developed the RAWSim-O simulation platform to emulate the RMFS warehouse. Compared to wave picking (Zhuang et al., 2021) (They say the orders are collected from 8 a.m. to 12 p.m., on average, 892 orders), Merschformann (Merschformann et al., 2019) decided on a real-time scale. The baseline of this research is the method

proposed by Merschformann (Merschformann et al., 2019). Two reasons are stated below:

1) When the wave size or time window is large, comparing real-time and wave size picking is unfair. That is because wave picking has more comprehensive information on orders.
2) Merschformann (Merschformann et al., 2019) proposed the method of dynamic order batching. That means the warehouse is running, and we must make POA and PPS decisions dynamically, depending on the current situation. Only partial orders are chosen from the order pool in each decision. Therefore, the left orders can match with the future arriving orders. That is unfair to the static problem that focuses on finishing all the orders in the order pool. The research solves POA and PPS in a dynamic environment, the same as what Merschformann did (Merschformann et al., 2019).

2.3 **Metaheuristic**

Compared to the exact method used to find global optimal in most cases, the metaheuristic is used to find a good quality solution to the NP-hardness problem in reasonable computational time (Blum & Roli, 2003). Most authors utilize metaheuristics to solve the order picking and batching problem, as mentioned in the previous section. Blum, C. and A. Roli (Blum & Roli, 2003) reviewed several metaheuristics with specific characteristics.

Diversity (exploration) and intensification (exploitation) are general and important metaheuristic concepts. Exploration explains the diversity of searching. Exploitation presents searching in a search place thoroughly. A comprehensive application of metaheuristics to a specific problem should consider these two points seriously. Neither being biased toward nor neglecting one side would harm the result. Some metaheuristics focus on neighborhood search to derive the solution. The structure of a neighborhood influences the ability and speed to find a global optimum. Several issues should be considered (Henderson et al., 2003). First, the size of the neighborhood is vital. If the size is too small, the solution's search space can reach its limit so that the convergence process is influenced. Besides, the solution is stuck in the local optimum easily. However, if the neighborhood size is too large, it is a random walk. The proper size of the neighborhood should be

decided according to the specific problem. The design of the function of the neighborhood is also essential (Eglese, 1990). A smooth searching space is requested rather than a rough one with lots of deep local optimum.

We utilize the Greedy Randomized Adaptive Search Procedure (GRASP) to solve our problem in this research (Feo & Resende, 1995). GRASP is a multi-start trajectory metaheuristic. The pseudo-code is listed following (Blum & Roli, 2003):

```
while termination conditions not met do
  s ← ConstrucrGreedyRandomizedSolution()
  ApplyLocalSearch(s)
  MemorizeBestFoundSolution()
endwhile
```

Figure 2-1 Pseudo code of GRASP

GRASP consists of two phases generally. First, it utilizes a constructive heuristic to generate an initial solution. The constructive heuristic starts with an empty partial solution. In each step, an item is added to the partial solution. A feasible solution is completed after several steps of construction. The difference between the traditional greedy method and GRASP is the utilization of the Restricted Candidate List (RCL). Compared to the pure greedy method of always choosing the best item in each step, the RCL is a mechanism to increase searching diversity. The function of RCL is stated below:

$$\bar{j} = \text{argmax}(f(j)|j \in \boldsymbol{P}) \quad \textbf{(2-4)}$$

$$\underline{j} = \text{argmin}(f(j)|j \in \boldsymbol{P}), \textit{where the } \boldsymbol{P} \textit{ is the search space of } j. \quad \textbf{(2-5)}$$

$$\text{RCL} = \left(f(\bar{j}) - f(\underline{j})\right) * \alpha + f(\underline{j}) \quad \textbf{(2-6)}$$

given P is the set consisting of all the j . j is the choice of item(in our case, the item is a pod). $f(j)$ represents how much the item j contributes to the objective function. $\bar{j}$ and $\underline{j}$ are the two items that contribute maximum and minimum values to the objective function, respectively. The RCL is created by three elements, which are $f(\bar{j})$, $f(\underline{j})$ and α. The parameter α is the controller of greediness in each step. $\alpha = 1$ means totally greedy. We can only select the item j, whose $f(j)$ is greater than RCL in each step. The second GRASP phase is local search, which aims to find the local optimum. Pitsoulis, L.S. and M.G. Resende (Pitsoulis & Resende, 2002) presented the importance of an initial solution in the local search. If the local search

starting from a good initial solution can finally result in a global optimum, it is regarded as the "basin of attraction of the global optimum." Once the reasonable initial solutions are generated repeatedly, it is expected that a good initial solution that can lead us to a global optimum can be found. That is the core concept of GRASP. Blum, C. and A. Roli (Blum & Roli, 2003) stated the situation that GRASP can be effective, which are

1) "The solution construction mechanism samples the most promising regions of the search space."
2) "The solutions constructed by the constructive heuristic belong to basins of attraction of different locally minimal solutions." We believe these conditions are met in our application of GRASP.

In this research, we use the Variable Neighborhood Descent (VND) as the structure of the local search in GRASP (Hansen et al., 2019). The pseudo-code of VND is shown in Figure 2-2. VND uses several neighborhoods to increase the diversity of searching when there is no improvement in the current one. In Figure 2.2, the $N_1 \dots N_k$ are the neighborhoods defined by the users. X is defined as the current solution. X' is defined as the best neighbor of X under a certain kind of neighborhood. If X' is better than X, we found a better solution in the given neighborhood. Hence, the X is updated by X'. Moreover, the k is updated to 1 to restart searching in the first level of the neighborhood. Otherwise, k is added one so that the searching in a new level of the neighborhood is started. With the changing neighborhoods, a search place that is "bad" given by one neighborhood originally may become a "good" search place given by another (Blum & Roli, 2003). "One operator, One Landscape" is the concept to describe those phenomenons (Jones, 1995a, 1995b).

```
function VariableNeighborhoodDescent (N_1 ... N_{k_max})
    repeat until no improvement or max CPU time elapsed
        k ← 1
        while k ≤ k_max:
            X' ← BestNeighbor(N_k(X))
            if objective(X') > objective(X):
                X ← X'
            else:
                k← k + 1
            endif
```

```
            endwhile
        endrepeat
endfuntion
```

Figure 2-2 Pseudo code of VND (Hansen et al., 2019)

CHAPTER 3
PROBLEM DESCRIPTION

3.1 System modeling

In this section, we present the model of the RMFS system related to our research. Because we only focus on optimizing POA and PPS, elements of replenishment and charging are ignored. Three levels of decision problems are modeled. At the strategic level, we decide the dimension of the area, pod batch, number of pods, and picking station placement. There are four horizontal aisles and five vertical aisles in the layout. There are 100 pods, and the pod batch is set 5 (horizontal) * 2 (vertical) equals 10. A single picking station case is considered in the research(see Figure 3-1).

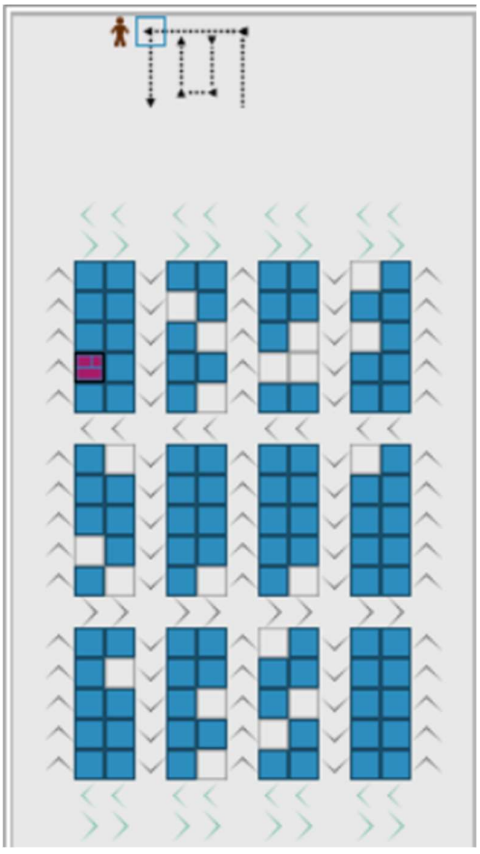

Figure 3-1 Layout of RMFS system

In the tactical level problem, SKU storage affects system throughput a lot. From the perspective of SKUs, the SKUs are classified into three classes according

to the demand. The more popular the SKU is, the more places(pods) it is put at. From the perspective of pods, every pod contains multiple classes of SKU.

In the operational level problem, “Order Cycle” and “AGV Cycle” are considered (see Figure 3-3 and Figure 3-2). These two cycles represent the related process flow of RMFS in this research. The Order Cycle makes the decisions on POA and PPS, which are the main topic of the research. The detailed explanation of each event in the Order Cycle is described as follows:

1) Check numbers of assigned pods:

   With the appropriate number of resources, there should be no idle workers, AGV, and bins in a cost-effective warehouse. The trigger for POA and PPS is set to keep pickers able to pick the SKU all the time. The system records the number of assigned pods, the pod selected in PPS. A predefined level π is set. Once the number of assigned pods is smaller than π, the system triggers the order picking process, solving POA and PPS. This research trivially sets π as the number of AGVs in the layout. That is because some AGVs do not have assigned pods to deliver when the number assigned is smaller than the number of AGVs. After the order picking process is triggered, the system solves POA and PPS with the proposed method. Some doubt it is too late to decide the pod to deliver when an AGV is available. Users can decide on an appropriate level of π by considering the algorithm's computational time and AGV turnover time. Again, the π is set trivially in this research to compare with the baseline.

2) Pick Order Assignment:

   Choose orders from the order pool and assign them to the picking station. Because of the constraint of bins, there are at most bin numbers of orders that can be processed parallelly. Therefore, the sequence of orders is also optimized in this research.

3) Pick Pod Selection:

   To choose the pods to fulfill the assigned orders in the picking station. Because of the constraint of bins, the sequence of pods visiting is also optimized in this research.

4) Generate and Store results in Pick Order Task Pool

   The results of the order picking process are saved in Pick Order Task Pool so

that AGVs can process later.

The AGV Cycle describes the AGV movements. It starts at AGV, leaving picking stations, and ends at AGV, heading to picking stations. The detailed explanation of each event in the AGV Cycle is described as follows:

1) Pod Storage Assignment:

   AGV must find empty spaces to store the current carried pods after picking in picking stations. Hence, the Pod Storage Assignment is needed. In this research, we utilize the shortest-distance policy. Therefore, the empty spaces with the shortest distance from the current location of pods are chosen for storage.

2) Path Planning:

   Path Planning is used to generate the routes of starting and ending points. In this research, it is used when AGV delivers carried pods from picking stations to designated empty space, AGV travel from designated empty space to assigned pods, AGV delivers assigned pods to designated picking stations. The A* algorithm is used for solving the Path Planning problem (Clarke & Wright, 1964).

3) Pick Order Task Allocation

   After the AGV stores the pods, they need to find assigned pods to deliver. The tasks in Pick Order Task Pool are assigned to these available AGVs. As we mentioned in Order Cycle, the PPS decides the sequence of pods visiting picking stations. Therefore, the Pick Order Task Allocation follows the result of PPS to assign available AGV to assigned pods. The first available AGV is assigned to the first pod in the pod sequence, generated in the PPS, the second available AGV is assigned to the second pod, and so on. Because there is no research studying optimizing the Task Allocation problem following the given order sequence and pod sequence, a simple heuristic is utilized in this research to test our POA and PPS model.

   As mentioned, the series of available AGVs would follow the pod sequence to identify its pods. After that, those AGVs go directly to their pods and stay under them. Here, we define the ***FA*** as the first AGV, representing the AGV taking the first pod in the pod sequence. ***SA*** is the second AGV, which means the AGV takes the second pod in the pod sequence. The distance from both ***FA*** and ***SA***

to the picking station is defined as d_{FA}^{p} and d_{SA}^{p} respectively. A safety-parameter β is set to avoid the wrong pod sequence happening. The rule is, that ***FA*** can always depart directly after it reaches its pods. The ***SA*** can make only departure when the following conditions meeting:

$$d_{SA}^{p} \geq \frac{d_{FA}^{p}}{\beta} \quad \textbf{(3-1)}$$

after the condition is met, the ***SA*** becomes ***FA,*** and we define the new ***SA*** as the AGV taking the pod positioned right after the original ***SA***. With this mechanism, we ensure the AGV can follow the pod sequence(our research focus) to deliver pods.

This research focuses on the POA and PPS decisions. The results are applied in the mentioned RMFS process flow. Our ultimate goal is to achieve higher throughput order. Other indicators in this research are recorded, including throughput SKU, pile-on value, and the average time orders occupying the bins. A baseline (Merschformann et al., 2019) is compared to the proposed POA and PPS model. A detailed description of the problem is stated in the next section.

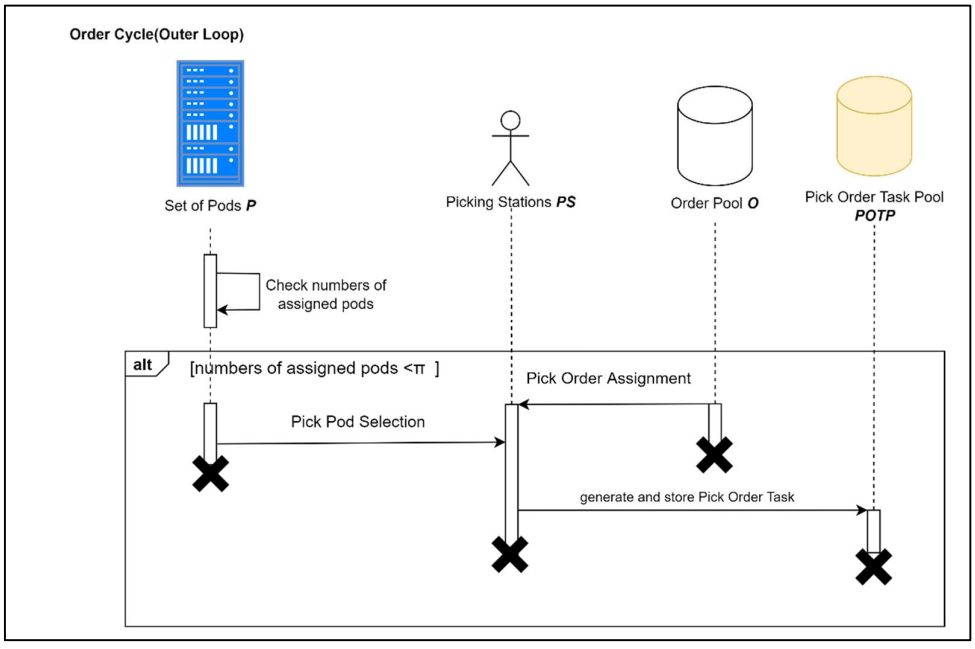

Figure 3-2 Sequence diagram of AGV Cycle

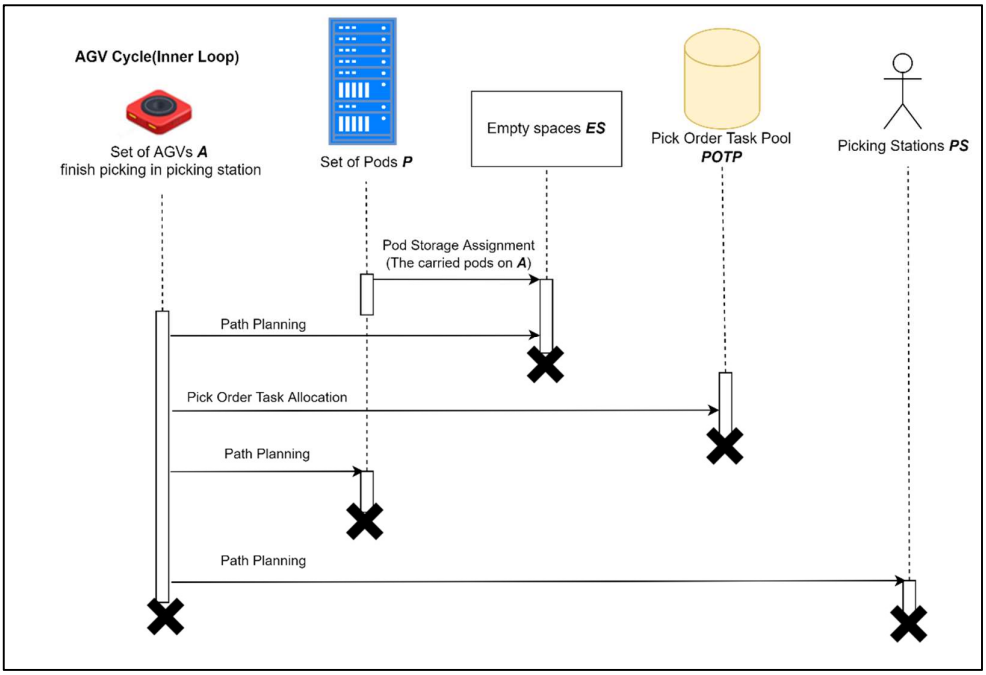

Figure 3-3 Sequence diagram of Order Cycle

3.2 **Problem description**

In this research, we assume that the orders arrive in the system endlessly and follow Poisson distribution at a rate λ per hour. Furthermore, we assume the system is in a stable status. That means (1) the resources, such as the number of AGVs, picking stations, and pods, are appropriate and always enough to fulfill the orders. (2) No dramatic peak demand happens like Double Eleven Shopping Festival.

This research assumes that each SKU in order has 1 quantity to simplify the problem. The pod scheduled to the picking station always has enough inventory to provide any quantity of SKUs it consists. This assumption is reasonable in a specific situation. That is, most of the final customers buy things in small quantities. (Boysen et al., 2017). As a result, the pile-on value definition in this research differs from Merschformann's (Merschformann et al., 2019). Merschformann's research considers the quantity issue

Table 3-1 Notation of mathematical model

Set	meaning
$\boldsymbol{O}$ {1....i}	The order pool consists of orders that have arrived in the system but have not been assigned
$\boldsymbol{P}$ {1....j}	The pods filled with SKUs used to fulfill the orders
$\boldsymbol{S}$	The set of SKUs demanded by all the orders.
$\boldsymbol{S_i \subseteq S}$	The SKUs demanded by order i
Parameter	meaning
$I_{j\in P,s\in S}$	Binary variable. 1 if pod j contains SKU s.
$Q_{i\in O}$	Nonnegative integer, number of SKUs in order i.
B	Number of bins in a picking station
L	Number of discrete time slots
θ	Completion bonus
Decision variable	meaning
$x^c{}_{i\in O}$	Binary variable, 1 if order i is chosen from $\boldsymbol{O}$ and can be completed.
$y_{i\in O,l\in L}$	Binary variable, 1 if order i is assigned to the picking station at discrete time slot l.
$z_{j\in P,l\in \mathrm{L}}$	Binary variable,1 if pod j visits the picking station at time slot l.
$\alpha_{\mathrm{i}\in \mathrm{O},s\in S,\mathrm{l}\in \mathrm{L}}$	Binary variable, 1 if SKU s for order i is provided at time slot l
$\gamma_{l\in L}$	Binary variable, 1 if the pods visit at time slot l and l-1 are different. That means the system has one more pod visit.

.

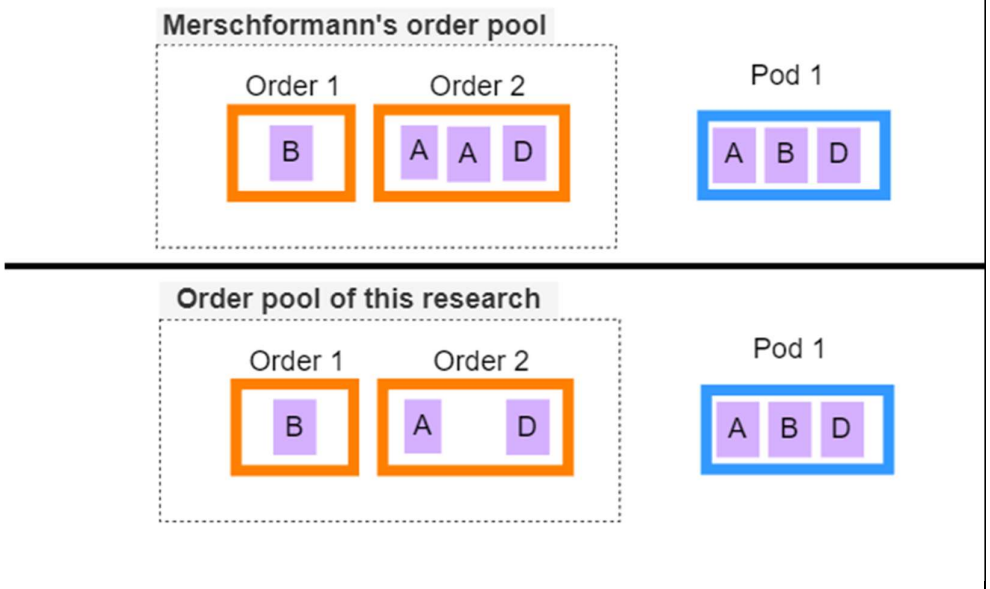

Figure 3-4 Example of difference of pile-on calculation

In Figure 3-4, the pile-on value of pod 1 of Merschformann's version is 4. That is because the pod 1 can provide 1 quantity of SKU B, 2 quantities of SKU A, and 1 quantity of SKU C. However, this research regards pile-on 3. That is because every SKU in order has only 1 quantity. Here, we define "pile-on quantity specific" and "pile-on types specific" to represent two definitions of pile-on from Merschformann and this research. Only the "pile-on types specific" is considered throughout the whole article.

Compared to literature optimizing a bunch of orders at a time (Boysen et al., 2017; Zhuang et al., 2021), this research shortens the time scope of the operational level decision to reply to the variant demand in a large assortment warehouse. However, when the start condition of the order picking process, *i.e.*, the number of the assigned pod is smaller than that of the AGV, is triggered, we have to derive the decisions mentioned in chapter 1.

Given a set of orders $\boldsymbol{O}$ and a set of pods $\boldsymbol{P}$, we need to find a set of selected orders $\boldsymbol{SO} \subseteq \boldsymbol{O}$ for the picking station and schedule a series of pod visits. Two objective functions are proposed.

$$Max \ \frac{\sum_{l\in \mathrm{L}} \sum_{\mathrm{i}\in \mathrm{O}} \sum_{s\in S_i} \alpha_{i,\mathrm{s},\mathrm{l}}}{\sum_{l=2}^{L} \gamma_l + 1} \quad \textbf{(3-2)}$$

$$Max \ \frac{\sum_{l\in \mathrm{L}} \sum_{\mathrm{i}\in \mathrm{O}} \sum_{s\in S_i} \alpha_{i,\mathrm{s},\mathrm{l}} + \sum_{i\in \mathrm{O}} x_i^c * \theta}{\sum_{l=2}^{L} \gamma_l + 1} \quad \textbf{(3-3)}$$

The objective function (3-2) maximizes the average number of SKUs that can be fulfilled per pod visit. This is so-called pile-on value(see equation (2-1)). The objective function (3-3) is also an interpretation of pile-on with an additional term. The term illustrates the number of orders completed among those pod visits. The occupation of bins inspires the idea of this term. An order occupying the bin for a long time decreases the picking station capacity. If we can process the order in each bin as fast as possible, then the next orders can be assigned to the available bins thick and fast. The parameter θ is the extent of the "completion bonus." When the θ is bigger, the solver prefers selecting the SKU (or pods) having the ability to complete orders compared to other choices only contributing SKU (fulfilling orders partially). For instance, assume θ is set three. There are two pods to choose from. One can provide one SKU, which can complete an order. The other pod provides two SKUs, but no orders can be completed. The objective value of choosing the first pod is 1+ 3 = 4. One of the later pods is only 1+1=2. Although the later pod brings more SKU, the first pod prevails it with the help of θ. That is because the first pod can "clean" a bin so that the following orders can be assigned to bins and fulfilled. A similar concept is also utilized in (Zhuang et al., 2021).

$$\sum_{j\in P} z_{j,l} \leq 1, \forall l = 1 \dots L \quad \textbf{(3-4)}$$

$$\sum_{i\in O} y_{i,l} \leq B, \forall l = 1 \dots L \quad \textbf{(3-5)}$$

$$\sum_{l=1}^{L} \alpha_{i,\mathrm{s},\mathrm{l}} \leq 1, \forall i \in O, s \in S_i \quad \textbf{(3-6)}$$

$$y_{i,\mathrm{l}} - x_i^c \leq y_{i,l'}, \forall l = 1 \dots L, \ l' = 1 \dots L, l < l' \quad \textbf{(3-7)}$$

$$2 * \alpha_{\mathrm{i},\mathrm{s},\mathrm{l}} \leq \ y_{i,\mathrm{l}} + \sum_{j\in \mathrm{P}} z_{j,l} * I_{j,s}, \forall i \in \boldsymbol{O}, \forall s \in \boldsymbol{S_i}, \forall l = 1 \dots L \quad \textbf{(3-8)}$$

$$\sum_{l=1}^{L} \sum_{s\in S_i} \alpha_{i,\mathrm{s},\mathrm{l}} \geq Q_i * x_i^c, \forall i \in \boldsymbol{O} \quad \textbf{(3-9)}$$

$$z_{j,l} - z_{j,l-1} \leq \gamma_l, \forall \, j \in \boldsymbol{P}, l = 2 \ldots L \quad \textbf{(3-10)}$$

$$x^c{}_i \in \{0,1\} \quad \textbf{(3-11)}$$

$$y_{i,l} \in \{0,1\} \quad \textbf{(3-12)}$$

$$z_{j,l} \in \{0,1\} \quad \textbf{(3-13)}$$

$$\alpha_{i,s,l} \in \{0,1\} \quad \textbf{(3-14)}$$

$$\gamma_l \in \{0,1\} \quad \textbf{(3-15)}$$

Constraint (3-4) presents that only one pod can be picked at every time slot l. Constraint (3-5) there are at most B number of orders in the picking station at every time slot l. Constraint (3-6) limits the SKU s demanded by order i can only be provided at most 1 time. Constraint (3-7) ensures the assigned order would occupy the bin until it has been completed. Constraint (3-8) ensures the system provides SKU s to order i only when the pods can provide SKUs visiting the picking station. Constraint (3-9) ensures x_i^c equals to one when order i is completed. Constraint (3-9) records the pod change. The rest of the constraints are the domain of decision variables.

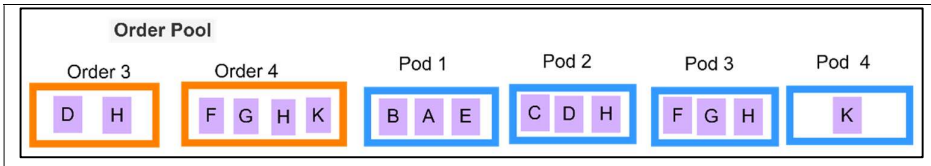

Figure 3-5 Example of order and pod sets

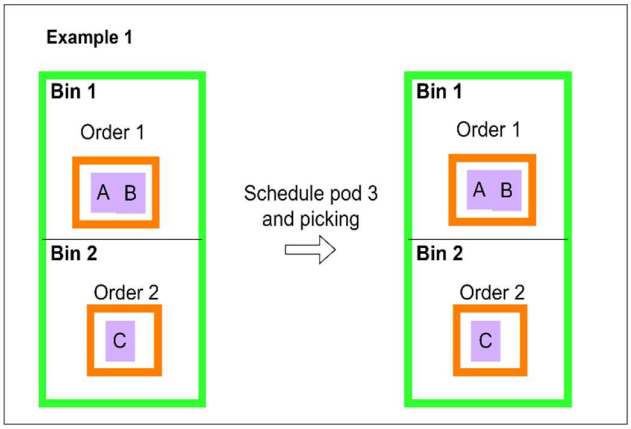

Figure 3-6 "Demand" policy for PPS

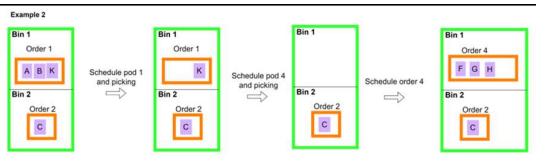

Figure 3-7 "Pile-on" policy for PPS

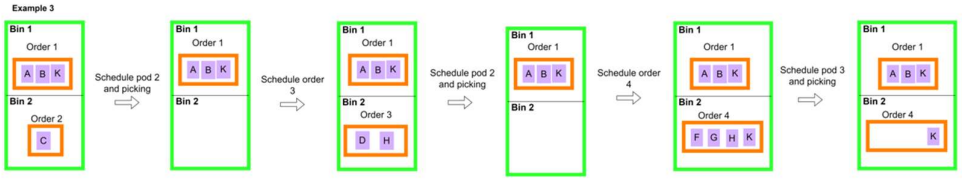

Figure 3-8 Consider orders in the order pool and bins when solving PPS

Figure 3-5 to Figure 3-8 shows three examples of different feasible solutions to the problem. The POA of three examples is all "Pod-match" policies (Merschformann et al., 2019). That means always assigning the orders can be provided the most SKU by the pods. The first example shows the reason not only to consider orders in the order pool when solving PPS. That is so called "Demand" policy for PPS (Merschformann et al., 2019). If we only match the orders in the order pool with pods, it shows that pod 3 can provide the most SKU to them. Therefore, we choose pod 3. However, the orders in the bins block pod 3 from providing SKU to picking stations. This example shows that only matching orders in the bins and pods is not the best choice. In the example 2, we assign the pod which can provide the most SKU to the orders in the picking stations. That is so called "Pile-on" policy for PPS (Merschformann et al., 2019). In the end, the number of fulfilled SKUs is 4, and the pod visits are 2. Therefore, the pile-on value is 4 / 2 = 2 (if we use equation (3-2)). In example 3, the solution fulfills 6 SKUs with 2 pods. The pile-on value is 6 / 2 = 3. As a result, the best solution is to schedule pods 2 and 3. The corresponding order sequence is order 3 and order 4. Although we know order 1 and 4 can be fulfilled by other pods in example 3. We hold the decision until next start condition of POA and PPS are met. It is possible that the future orders match them well, that results in a higher pile-on (making better use of pod 1). At least, there is no benefit to schedule the pods so early when no AGV eager for it.

3.3 **Heuristic method**

As mentioned in (Zhuang et al., 2021), the POA and PPS problems are NP-hard. It is not efficient to use the exact method to solve the problem. Hence, the metaheuristic GRASP is applied to derive the solution efficiently and effectively. In this section, we first introduce how to generate the initial solution. After that, the local search is conducted to improve the initial solution. The whole construction starts from a picking station with bins full of partial orders. It is reasonable to fill the available bin right after an order is completed so that no bins are idle. Therefore, there are no empty bins before or after a pod visit.

The procedure of choosing a pod to process these orders in the bin is twofold. In the first step, the contribution of each candidate pod $\mathrm{j}, \forall \mathrm{j} \in \boldsymbol{P}$, defined by $f(\mathrm{j})$, is calculated. Then, candidate pods $\bar{\jmath}$ and $\underline{j}$ of which $f(\bar{\jmath})$ and $f(\underline{j})$ are maximal and minimal across all $f(\mathrm{j})$, i.e., $\bar{\jmath} = \operatorname{argmax}(\{f(\mathrm{j}) \mid \forall \mathrm{j} \in \boldsymbol{P}\})$ and $\bar{\jmath} = \operatorname{argmin}(\{f(\mathrm{j}) \mid \forall \mathrm{j} \in \boldsymbol{P}\})$, are used to obtain the RCL value in equation (2-6). In the second step, one of the candidate pods of which $f(\mathrm{j})$ is greater than RCL is randomly chosen to process the orders in the bins. The chosen candidate pod is scheduled in the pod sequence. We repeat the above constructive heuristic until N pods are chosen. The parameter N determines how many pods being scheduled in the initial solution. The larger N increases the diversity of searching but also increases the computational time. Then, an initial solution associated with the order and pod sequence can be generated.

In fact, the calculation of $f(j)$ implies the scheduling of orders. That can be divided into two steps. The first step is calculating the fulfillment of original partial orders in the bin. After that, we can imagine some of them are completed so that available bins are released. The second step is to assign new orders to those newly available bins. The orders which can be completed by the candidate pod solely are scheduled first. We define this type of order as "Single-pod orders." Conversely, those orders that the candidate pod cannot complete are defined as "Multi-pod orders." We have to schedule one Multi-pod order for each newly available bin so the bins are full after the candidate pod leaves. Considering the choice of Multi-pod orders, we directly schedule the Multi-pod order that can contribute the objective value the most corresponding to the candidate pod. Figure 3-9 illustrates an example

of calculation of $f(j)$ and scheduling of orders.

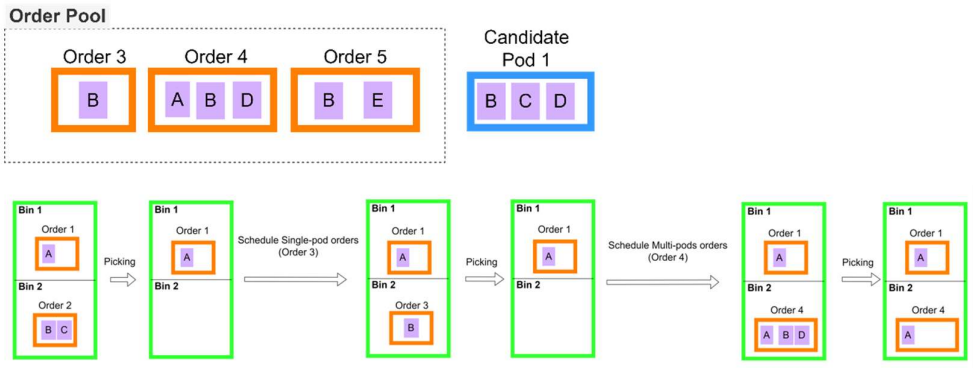

Figure 3-9 Example of calculating objective value caused by candidate pod 1, which is denoted as $\boldsymbol{f(candidate\ pod\ 1)}$

In this example, candidate pod 1 completes the original partial order 2 in the bin. Bin 2 is free, so new orders must be assigned and scheduled. As mentioned, the Single-pod orders are scheduled first. Then, order 3, which can be completed by candidate pod 1, is scheduled. After all the Single-pod orders are scheduled, we have to schedule Multi-pods orders. In this case, order 4 and order 5 are the alternatives. Order 5 can only contribute 1 SKU, but order 4 contributes 2 SKU. Therefore, order 4 is scheduled. The ending bin status shows the partial orders left if we schedule candidate pod 1. To sum up this example:

- if we schedule candidate pod 1, the consequent order sequence: order 3 -> order 4 is scheduled.
- If we use objective function (3-2), $\boldsymbol{f(candidate\ pod\ 1) = 5}$
- If we use objective function (3-3), 2 orders are completed. Therefore, $\boldsymbol{f(candidate\ pod\ 1) = 5 + 20}$

After generating an initial solution, we start to conduct a local search to improve it. The concept of local search of order picking problem in RMFS is from (Boysen et al., 2017). The author suggests generating a new order sequence and then a new pod sequence based on the new order sequence. That has a better result than conversely (new pod sequence → new order sequence). Therefore, a similar search is conducted in this research. The following important issue would be the operator to generate the new order sequence. As mentioned in chapter 2.3, the VND

is utilized as the structure of local search in this research. Considering the computational time and effectiveness, two neighborhoods are defined to generate a new order sequence. The first neighborhood N_1 is defined by switching two orders in the order sequence arbitrarily and find the pod sequence by beam search. This neighborhood is to strengthen the intensification of searching. The searching in N_1 is conducted until no more improved solution can be found. After that, the searching in the second neighborhood N_2 is conducted. Before the introduction of N_2, we define "potential orders."

Table 3-2 Notations for local search

Set	meaning
$\boldsymbol{PO}$ $\{1 \ldots po\}$	Set of potential orders, which can be fulfilled at least one SKU by scheduled pods.
$\boldsymbol{S_{po}}$	The SKU demanded by order po.
$\boldsymbol{SP}$	The set of scheduled pods.
$\boldsymbol{S_{sp}}$	The set of SKU can be provided by scheduled pods.
$\boldsymbol{MPO}$ $\{1 \ldots mpo\}$	Set of Multi-pods orders scheduled in the order sequence.
Parameter	**meaning**
$PQ_{i \in O}$	Nonnegative integer, number of SKU can be provided by scheduled pods in order i.
$EQ_{i \in O}$	Nonnegative integer, number of left SKU to be fulfilled in order i.

$$PQ_{po} = \left|S_{po} \cap S_{sp}\right|, \forall po \in \boldsymbol{PO} \quad \textbf{(3-16)}$$

$$p(po) = \frac{PQ_{po}}{\sum_{po \in \boldsymbol{PO}} PQ_{po}} \quad \textbf{(3-17)}$$

$$g(mpo) = \begin{cases} Q_{mpo} + \theta, if\ \ EQ_{mpo} = 0\ and\ objective\ function\ (3\text{-}3)\ \text{is used} \\ Q_{mpo} - EQ_{mpo}, \text{O. W.} \end{cases}, \forall mpo \in \boldsymbol{MPO} \quad \textbf{(3-18)}$$

$$\overline{mpo} = argmax(g(mpo) | mpo \in \boldsymbol{MPO}) \quad \textbf{(3-19)}$$

$$p(mpo) = \frac{g(\overline{mpo}) - g(mpo)}{\sum_{mpo \in MPO} g(\overline{mpo}) - g(mpo)}, \forall mpo \in \boldsymbol{MPO} \quad \textbf{(3-20)}$$

The potential orders are the orders that can be fulfilled (or even completed) by the scheduled pods in the pod sequence of the initial solution. This idea comes from the baseline (Merschformann et al., 2019). They propose a "pod-match" policy in POA. When the system needs to assign an order to the picking station, this policy chooses the one matching the pods heading to the picking station the most. In our

constructive heuristic, we schedule one pod at a time. Therefore, The pods coming(scheduling) later are not considered when scheduling orders. That may result in some multi-pod orders not matching the pod scheduled later. The scheduling of potential orders can improve the mentioned drawback and enhance the diversity of searching. Therefore, N_2 is defined by switching two orders selected from the order sequence and potential orders and find the pod sequence by beam search. The roulette wheel selection, which is utilized in genetic algorithms usually (Mitchell, 1998), is also employed in the selection of two orders in N_2.

Equation (3-16) calculates the number of SKUs that the scheduled pod can provide for the selection of potential orders. After that, the probability of the chosen potential order is calculated by equation (3-17). The selection of Multi-pod order is derived from equation (3-16) to (3-20). As mentioned before, the bins are always full after the pod leaves. Hence, SKUs are left to be fulfilled after the pod visits. Those left SKUs are used to calculate the worth of the Multi-pod order (see equation (3-18)). The more SKUs left, the less it has been fulfilled. Then, it is more likely to be selected to switch with potential orders. It is noted that the Multi-pod order contributing "smaller" objective value has priority in being selected. Therefore, we use equation (3-19) to find the best Multi-pod and use its objective value to subtract others (see equation (3-20)). This way, the smaller one has a higher probability of being selected.

After the operator is executed, we get a new order sequence. The next step is to find a new pod sequence based on the new order sequence. Boysen et al. (Boysen et al., 2017) have done it with beam search. In this research, we follow their approach with some modifications. Beam search (*Introduction of Beam Search*) is a breadth-first and best-first searching method. That means all the nodes in the same level are branched (breadth-first), and the only best beam width numbers of nodes are kept (best-first). In this research, each node in the beam tree has four elements:

1) The bins status shows which order is occupying the bin currently.
2) The order sequence shows the orders assigned to the newly available bin.
3) Pod sequence records the pods that have been scheduled during the beam search.
4) Objective value records the objective value caused from root to current node.

For each node, we branch by all the candidate pods. It is noted that the candidate pods are those scheduled in the initial solution instead of ***P*** to reduce the searching time. Then, for each branch, we update the pod sequence, schedule the orders following the order sequence and calculate the objective value according to the situation of the fulfillment of orders. After calculating all the branches in the same depth, the top beam width number of branches by objective value is kept.

A branch is finished when its objective value doesn't improve from its parent node → its parent node becomes a solution and is recorded in the solution list. It is noted that the input order sequence may be pruned or expanded. If the order sequence is pruned, the first few orders generate a greater pile-on. Adding the other orders would lead to a decrease in pile-on. If the order sequence is expanded, the modified order sequence makes some orders completed, and some bins are released and available for more orders. For those newly available bins, we use the greedy method to assign the orders which can be fulfilled by the current pod the most(still, assign all Single-pod orders and one Multi-pods order). After all the branches are finished, we select the one having the largest objective value in the solution list. With this method, the new pod sequence is created. The new solution, which consists of the new order generated by the operator and the new pod sequence created by beam search, is accepted when it has a larger objective value. Otherwise, we use the original order sequence to conduct the next iteration of the local search. To determine the local optimum, the parameter ρ is set. If there is no improvement in ρ number of continual iterations, the current solution is set local optimum. Figure 3-10 shows the pseudo-code of the proposed heuristic.

```
Initialize GlobalBest
While termination conditions not met do:
    #Generate initial solution
    Initialize X[order sequence] # empty parital solution
    Initialize X[pod sequence] # empty partial solution
    While number of scheduled pods < N do:
        equation ( 2-4) # Find pod
        equation ( 2-5)
        equation ( 2-6)
        j* ← random choose a pod j ,where f(j) is larger than RCL
        schedule pod j* to X[pod sequence]
        schedule corresponding orders to X[order sequence]
```

```
endwhile
# Local search
k ← 1
ρ_count ← 0
while k ≤ k_max do:
    while ρ_count < ρ do:
        X' ← N_k(X)
        if objective(X') > objective(X):
            X ← X'
            ρ_count ← 0
            Improvement ← True
        else:
            ρ_count ← ρ_count + 1
        endif
    endwhile
    if Improvement = True:
        k ← 1
    else:
        k← k + 1
    endif
endwhile
update GlobalBest
```

Figure 3-10 Pseudo code of proposed heuristic

CHAPTER 4
EXPERIMENT VALIDATION

In this chapter, we conduct two experiments. The first experiment validates the proposed heuristic method's effectiveness with BARON solver, a commercial solver aiming at solving non-linear mixed integer programming models. The second experiment verifies if the proposed model can achieve higher throughput order than the baseline (Merschformann et al., 2019).

The multi-line order data is generated. The number of SKUs in each order follows truncated geometric distribution with p = 0.4 from 1 to |SKU| (Xie et al., 2021) (see Figure 4-1). The probability of each SKU follows the mass function of Geometric distribution with p = $\frac{5}{|SKU|}$ (Xie et al., 2021) (Figure 4-2). The SKUs are classified into 3 classes. The total number of pods and SKUs is 100 and 460, respectively. The ratio of the number of pods and SKU are set following the real industry. The top 10% of high-demand SKUs are classified as class A. The later 30% demand is defined as class B. The others are defined as class C. A mixed classes storage policy is used. Each pod has three classes. Class A has 2 types of SKUs, class B has 3 types of SKUs, and class C has 3 types of SKUs in a pod. That results in each type of class A SKU is stored in 2 * 100 / 46 = 4.34 pods, which accounts for 4.34 / 100 = 4.3% of total pods. With the same calculation, each type of class B and C SKU are stored in 2.17 and 1.63 pods and account for 2.17% and 1.63% of total pods. All the related settings are shown in Table 4-1.

Table 4-1 Experiment setting

Parameter	Value
Simulation	
Run time	1 hour
Replication	30
Layout	
Inventory Area	120 storage locations
Inventory Capacity	100 pods(83% of total)
Empty Storage	20 empty storage(17% of total)

Parameter	Value
Pod Batch	2 * 5 blocks
Aisles	4 horizontal aisles and 5 vertical aisles
Picking Stations	1 picking station
Orders	
Order line	Multi-line orders(multiple SKU, 1 quantity per order)
Initial Orders	50 orders
Arrival rate	Poisson's distribution with $\lambda = 2$ per 23 seconds.
SKUs	
Number of SKUs	460
Number of SKU in an order	Truncated geometric distribution with p = 0.4 from 1 to |SKU|(Xie et al., 2021)
SKU distribution	Mass function of Geometric distribution with $p = \frac{5}{\|SKU\|}$. (Xie et al., 2021)
SKU-to-Pod distribution	Class A: 2 SKU
	Class B: 3 SKU
	Class C: 3 SKU
AGVs	
Number of AGVs	5 AGVs
AGV Speed	1 m/s
Max Velocity with Pod	2
Max Velocity with No Pod	2.5
Turn Time 90 Degree	2.5 s
Turn Time 180 Degree	3 s
Lift Pod Time	4 s
Picking stations	
Picking time	Gamma distribution, $\alpha = 1$, $\beta = 1$

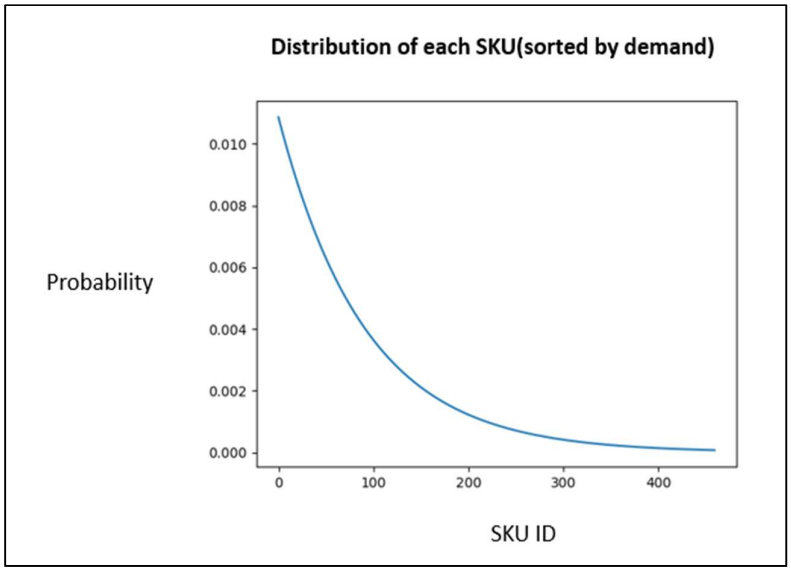

Figure 4-1 The distribution of each SKU

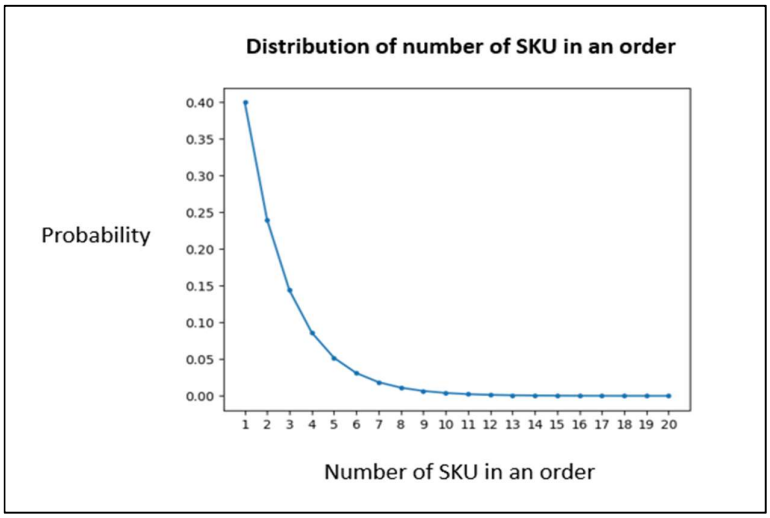

Figure 4-2 The distribution of the number of SKUs in an order

4.1 **Validation of proposed heuristic compared with a commercial solver**

We choose a certain number of orders from the order pool to compose a smaller order pool. In the meantime, the subset of pods that cover the demand SKU for the smaller order pool is selected in this experiment. The pods having the same pattern of SKUs are also eliminated. That is to reduce the search space. Otherwise, finding the solution with the solver using the exact method is challenging. Table 4-2 shows the small instances of the experiment. In the five orders cases, the solver can find the optimal solution within an hour. The proposed method can also derive

the optimal solution within one second.

The solver couldn't find the optimal solution within three hours in the 10 orders cases. One of the cases, consisting 10 orders, 20 SKU, and 19 pods, needs 1021 variables and 3122 linear constraints. We speculate that the exact method takes lots of time to derive the optimal solution because of its size and nature. Although the optimality is not proved, the proposed heuristic derives the better solution within a second. According to the result, utilizing the heuristic is at least an efficient way to solve the proposed model.

Table 4-2 Comparison between solver and proposed heuristic

		Baron solver		Proposed heuristic	
orders	bin	Opt(* optimal)	Time (sec)	opt	time
5	1	1.33*	123.6933	1.33*	0.01
	3	1.33*	259.0333	1.33*	0.01
10	1	1.55	More than 3 hours	1.83	0.03
	3	2.125	More than 3 hours	2.33	0.04
	5	2.13	More than 3 hours	2.33	0.057

4.2 **Validation of the proposed model**

This research proposes a real-time scale model to maximize the pile-on value. The ultimate goal is to pursue high throughput order. We conduct a simulation to emulate the real situation in the industry to prove the model's effectiveness and appliance. Because the data is usually large in the industry, we only use the proposed heuristic method to derive the solution instead of the commercial solver using the exact method. Furthermore, this research focuses on POA and PPS, so we only compare the methods to solve these two decision problems. Table 4-3 shows the related mechanism and setting of our proposed method and baseline.

Table 4-3 Settings related to proposed method and baseline

		Proposed method	**Baseline(Merschformann et al., 2019)**
POA	**Start condition**	When the number of assigned pods < number of AGV.	When there is an available bin.
	The scope	Dynamically depending on the orders in the order pool and the SKU combination in the pods.	Assign one order at a time.
	Method description	Use GRASP hybrid with VND to schedule an order sequence. (Refer to chapter 3.3)	Choose the order that can be provided the most SKU by the pods heading to the picking station.
PPS	**Start condition**	When the number of assigned pods < number of AGV. Actually, it is the same as that of the baseline.	When the AGV needs to find a pod to deliver.
	The scope	Dynamically depending on the orders in the order pool and the SKU combination in the pods.	Assign one pod at a time
	Method description	Use GRASP hybrid with VND to schedule a pod sequence. (Refer to chapter 3.3)	Choose the pod that can provide the most SKU to the bins.
Other setups		$\alpha = 0.8, \rho = 30,$ $\theta = 5, N = 10$	None

We simulate the warehouse operation in 1 hour following the process flow of RMFS (see chapter 3.1). The replication is set to 30 to avoid bias. In each replication, several indicators are recorded, including throughput order, pile-on order, throughput SKU, pile-on SKU, and average order occupying bin time. The

order throughput, which is our ultimate goal, indicates the total number of completed orders. The pile-on order is the number of throughput orders divided by that of the total pod visit. The throughput SKU is the total number of SKUs being picked. The pile-on SKU is the number of throughput SKUs divided by that of the total pod visit. Finally, the average order occupying bin time is the time a bin is occupied by an order averagely. It is intuitively that the longer an order occupies the bin, the fewer orders this bin can process. That also corresponds to the idea of the objective function (3-3). Therefore, we expect this indicator has a positive correlation with throughput order. The related analysis is presented in later sections.

The orders arrive in the system following Poisson distribution with $\lambda = 2$ per 23 seconds. 5 AGVs are used, and the detailed movements are considered, including moving speed with or without pods, acceleration, deceleration, turning, and lift pod time. The number of AGVs is set according to the capacity of the picking station in the real industry. That means there are at most 5 AGVs queuing in a picking station. Those AGV detailed movements are the real industry setting. The layout has 120 storage locations, and 100 pods are used. 17% empty for the flexibility of pod storage. The pod batch is set 2 * 5. There are four horizontal and five vertical aisles. The single picking station is placed on the long side. The picking time is set gamma distribution $(\alpha = 1, \beta = 1)$ per picking quantity. All the experiments are implemented in the agent-based simulation on the NetLogo platform. The proposed heuristic method is programmed in Python. Figure 4-3 shows the system framework.

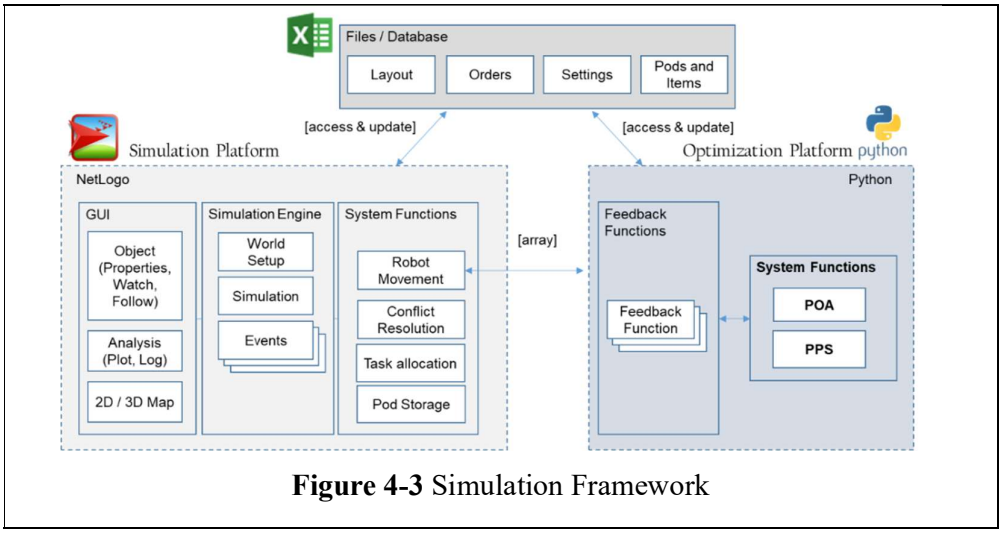

Figure 4-3 Simulation Framework

Figure 4-4 to Figure 4-8 are the results of the experiments. It is noted that the blue (with completion bonus) and orange (SKU pile-on) bar in those figures represents the utilization of objective function (3-2) and (3-3) in the proposed heuristic, respectively. To prove the significance of improvement of our proposed method, we conducted Tukey Honestly Significant Difference test (Tukey HSD test). The statistic test is used to check the pairwise difference among factors. First, our proposed method (with both objectives) has a significantly better result on every indicator (see Table 4-4 to Table 4-8). Purely optimizing pile-on SKU has an 7.4% higher throughput order than the baseline. With the help of the completion bonus, the gap is around 9%. These can be attributed to the direction of optimization in this research, which maximizes pile-on order and SKU. As shown in Figure 4-4, our proposed method can reach higher pile-on order. The difference between the completion bonus and baseline is around 0.22. In this case, our proposed method can complete one more order than the baseline every 4.6 pod visits on average.

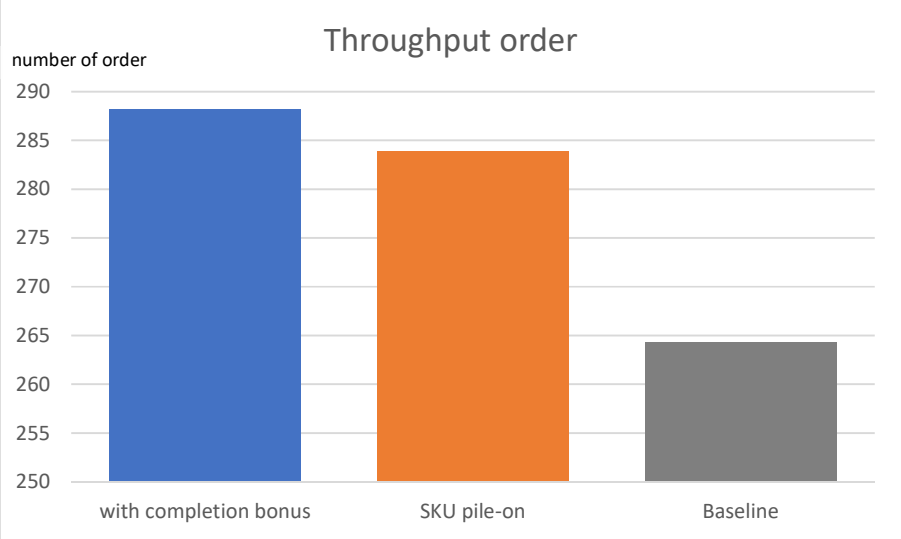

Figure 4-4 Throughput order

Table 4-4 Tukey HSD Test for throughput order

group 1	*group 2*	*mean*	*std err*	*q-stat*	*lower*	*upper*	*p-value*	*mean-crit*	*Cohen d*
with completion bonus	SKU pile-on	4.23333	0.99398	4.25894	0.88168	7.58498	0.00944	3.35165	0.777574
with completion bonus	Baseline	23.8333	0.99398	23.9775	20.4816	27.1849	3.93E-14	3.35165	4.37768
SKU pile-on	Baseline	19.6	0.99398	19.7185	16.2483	22.9516	3.93E-14	3.35165	3.600106

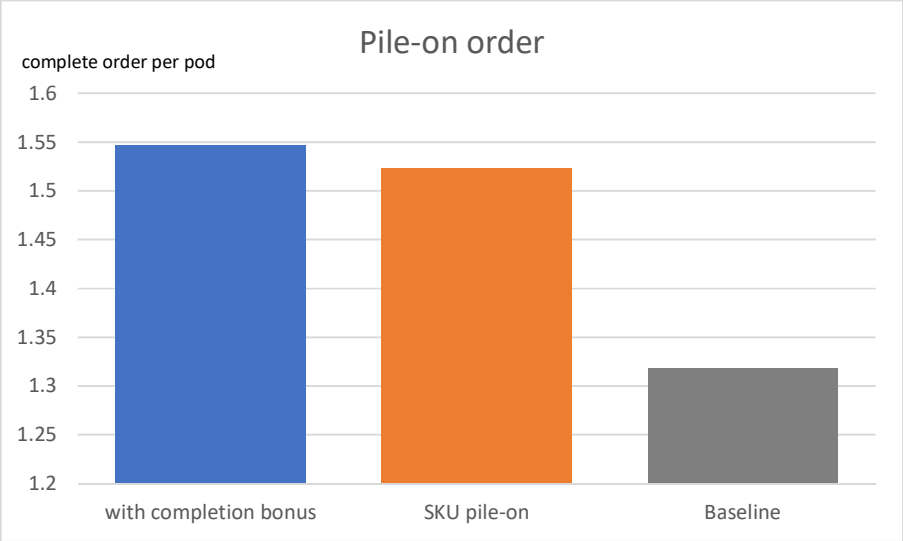

Figure 4-5 Pile-on order

Table 4-5 Tukey HSD Test for pile-on order

group 1	*group 2*	*mean*	*std err*	*q-stat*	*lower*	*upper*	*p-value*	*mean-crit*	*Cohen d*
with completion bonus	SKU pile-on	0.02403	0.005588	4.300284	0.005188	0.042873	0.008671	0.018843	0.785121
with completion bonus	Baseline	0.228682	0.005588	40.92306	0.209839	0.247525	3.93E-14	0.018843	7.471494
SKU pile-on	Baseline	0.204652	0.005588	36.62277	0.185809	0.223494	3.93E-14	0.018843	6.686373

With regard to throughput SKU, purely maximizing pile-on SKU and completion bonus prevail baseline around 2% and 1.6%, respectively. The higher throughput SKU leads to a higher utilization rate of the picking station. That is because the picker can only be idle or pick the item. The more the SKU is picked, the less idle the picking station is. The gap between purely maximizing the pile-on SKU and baseline on the pile-on SKU indicator is around 0.33. On average, every three pods can contribute one more SKU to the picking station.

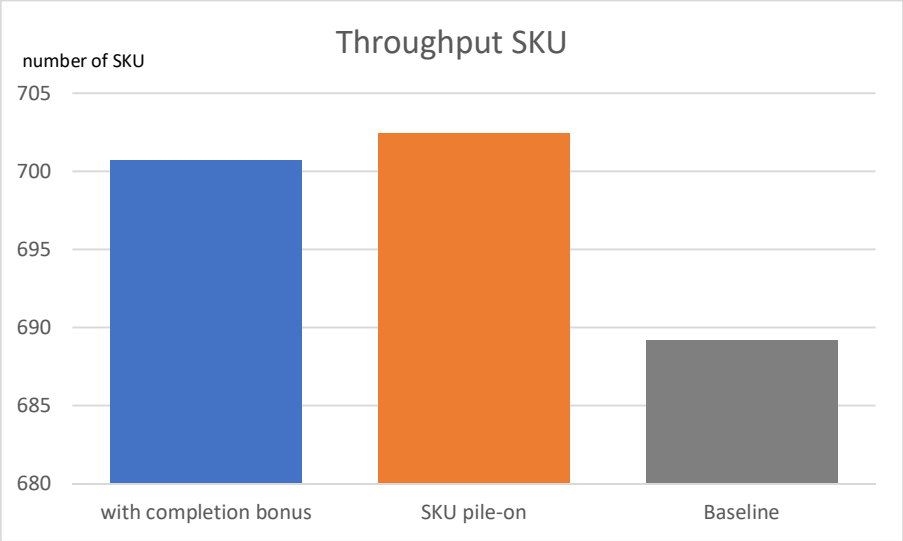

Figure 4-6 Throughput SKU

Table 4-6 Tukey HSD Test for throughput SKU

group 1	*group 2*	*mean*	*std err*	*q-stat*	*lower*	*upper*	*p-value*	*mean-crit*	*Cohen d*
with completion bonus	SKU pile-on	1.7	1.407429	1.207876	-3.04575	6.445753	0.670508	4.745753	0.220527
with completion bonus	Baseline	11.53333	1.407429	8.194613	6.787581	16.27909	3.21E-07	4.745753	1.496125
SKU pile-on	Baseline	13.23333	1.407429	9.402489	8.487581	17.97909	7.46E-09	4.745753	1.716652

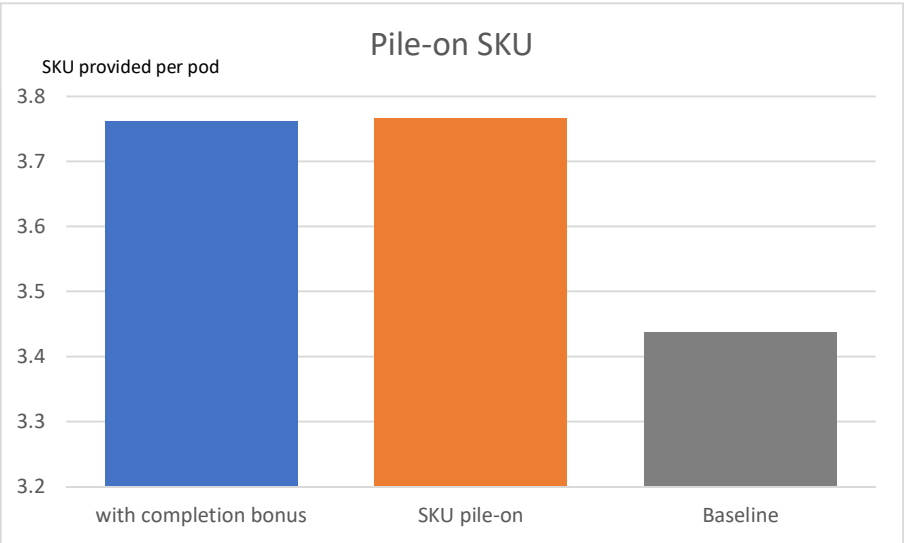

Figure 4-7 Pile-on SKU

Table 4-7 Tukey HSD Test for pile-on SKU

group 1	*group 2*	*mean*	*std err*	*q-stat*	*lower*	*upper*	*p-value*	*mean-crit*	*Cohen d*
with completion bonus	SKU pile-on	0.005777	0.009702	0.595464	-0.02694	0.038493	0.907014	0.032716	0.108716
with completion bonus	Baseline	0.324632	0.009702	33.45889	0.291917	0.357348	3.93E-14	0.032716	6.10873
SKU pile-on	Baseline	0.33041	0.009702	34.05435	0.297694	0.363126	3.93E-14	0.032716	6.217446

It is interesting to see the comparison between two proposed objective functions. The completion bonus could complete more orders. Conversely, the modestly superior throughput SKU is derived by pure optimization of pile-on SKU. The trade-off between throughput rate and completion of orders is the issue we face here. To match more SKUs in the bins, some orders with "rare" SKUs, *i.e.*, class C SKU, in the bins tend to be ignored. However, the situation happens less when we use a completion bonus. That is because the parameter θ reinforces the behavior of finishing orders. Once completion of orders in the bins is in priority, it is intuitive that the average order occupying the bin time would be shorter. Therefore, it is expected that the order occupying bin time of completion bonus is shorter than that of purely maximizing pile-on SKU. The similar result is shown in Figure 4-8.

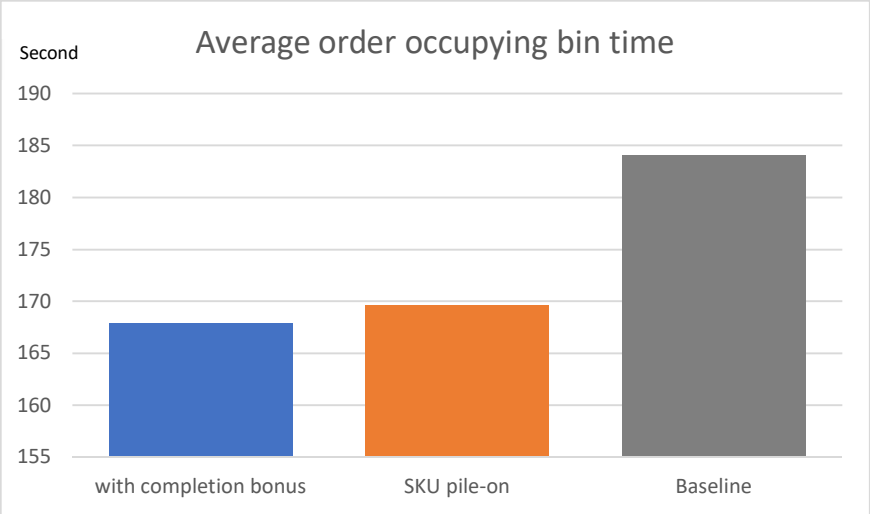

Figure 4-8 Average order occupy bin time

group 1	*group 2*	*mean*	*std err*	*q-stat*	*lower*	*upper*	*p-value*	*mean-crit*	*Cohen d*
with completion bonus	SKU pile-on	2.18006	0.64671	3.371001	0.000647	4.359474	0.049933	2.179413	0.60545
with completion bonus	Baseline	15.52065	0.64671	23.99939	13.34124	17.70006	2.02E-13	2.179413	4.310418
SKU pile-on	Baseline	13.34059	0.64671	20.62839	11.16118	15.52	2.02E-13	2.179413	3.704968

Table 4-8 Tukey HSD Test for average order occupy bin time

We conduct the regression analysis on the indicators. The regression analysis shows how the independent variable (X) influences the dependent variable (Y). In our case, we set order occupying bin time, pile-on order, pile-on SKU, and throughput SKU as independent variables and the throughput order as a dependent variable. First, all the results are significant (see Table 4-9 to Table 4-20). Figure 4-9 shows the relationship between order occupying time and throughput order. The R square, which shows the extent of the explanation of throughput by order occupying bin time, is 0.9915. The coefficient is -1.5012. That means the throughput order reduces 1 unit as the average order occupying time increases by around 1.6 seconds in this case.

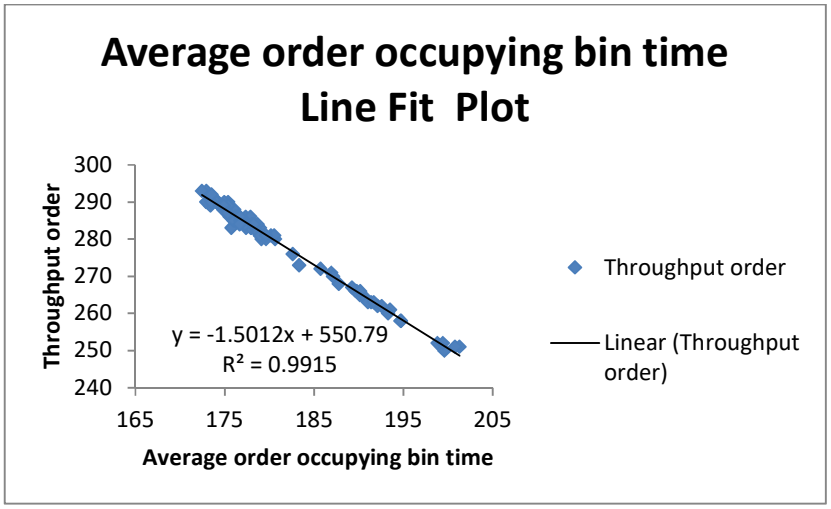

Figure 4-9 The fit plot of Throughput order (Y) and average order occupy bin time (X)

Table 4-9 Regression statistics of Figure 4-9

Regression Statistics	
Multiple R	0.995763
R Square	0.991543402
Adjusted R Square	0.991447305
Standard Error	1.086305416
Observations	90

Table 4-10 ANOVA test of Figure 4-9

	df	SS	MS	F	Significance F
Regression	1	12175.94	12175.94	10318.08	5.33184E-93
Residual	88	103.8452	1.180059		
Total	89	12279.79			

Table 4-11 The significance of the coefficient and intercept of Figure 4-9

	Coefficients	Standard Error	t Stat	P-value	Lower 95%	Upper 95%	Lower 95.0%	Upper 95.0%
Intercept	550.788581	2.67997252	205.520	8.2E-120	545.4626	556.1144	545.46269	556.11446
Order occupying bin time	-1.50118108	0.01477861	-101.578	5.33E-93	-1.53055	-1.471811	-1.5305504	-1.471811

Figure 4-10 shows the association between pile-on order and throughput order. The R square of it is only 0.9165. The coefficient presents that every increase

of 0.1 pile-on order leads to the improvement of 10 throughput orders.

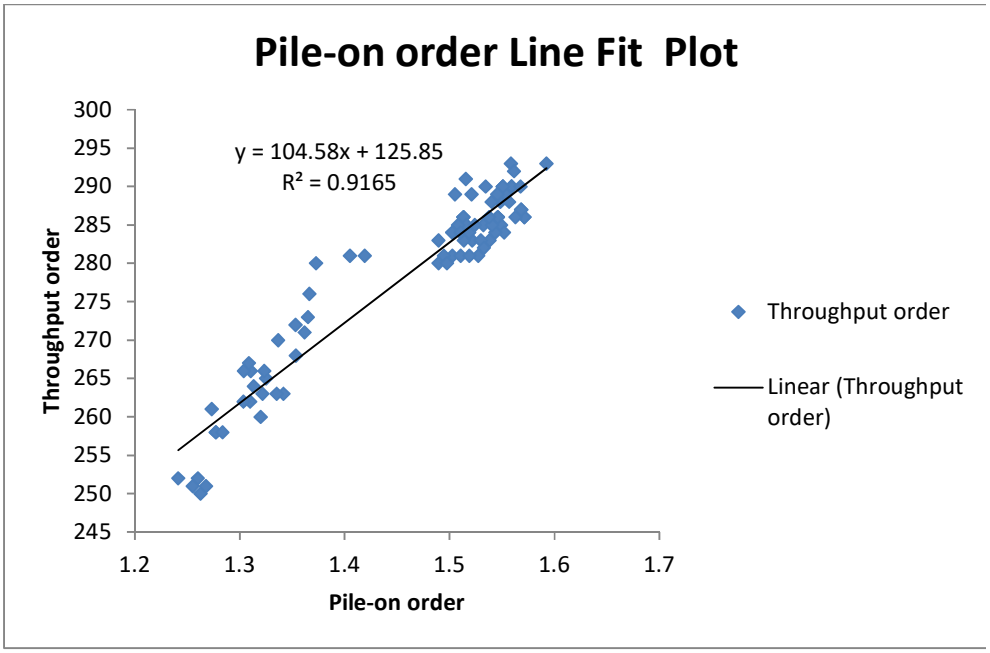

Figure 4-10 The fit plot of Throughput order (Y) and pile-on order (X)

Table 4-12 Regression statistics of Figure 4-10

Regression Statistics	
Multiple R	0.957322
R Square	0.916465
Adjusted R Square	0.915516
Standard Error	3.414197
Observations	90

Table 4-13 ANOVA test of Figure 4-10

	df	SS	MS	F	Significance F
Regression	1	11254	11254	965.4497	3.23E-49
Residual	88	1025.793	11.65674		
Total	89	12279.79			

Table 4-14 The significance of the coefficient and intercept of Figure 4-10

	Coefficients	Standard Error	t Stat	P-value	Lower 95%	Upper 95%	Lower 95.0%	Upper 95.0%
Intercept	125.8464	4.9361	25.4951	2.08E-42	116.0369	135.6558	116.0369	135.6558
Pile-on order	104.5766	3.365655	31.07169	3.23E-49	97.88805	111.2651	97.88805	111.2651

Moreover, Figure 4-11 illustrates the influence of pile-on SKU on throughput order. The R square of it is 0.91. Throughput orders are improved by around 5 when the pile-on SKU increases by 0.1. Finally, Figure 4-12 presents the impact of throughput SKU on throughput order. The R square is 0.7149, which is the lowest one compared to others. Every 1 more throughput SKU causes around 1 more throughput order. Following the results, we discover that the highest correlation is between order occupying bin time and throughput order. The pile-on value performed worse because it is influenced by the number of pod-visit. Hence, we can see some data points have similar pile-on but different throughput orders. Compared to it, order occupying bin time is more robust. To sum up, we infer that the depreciation of order occupying bin time can increase the throughput order potentially.

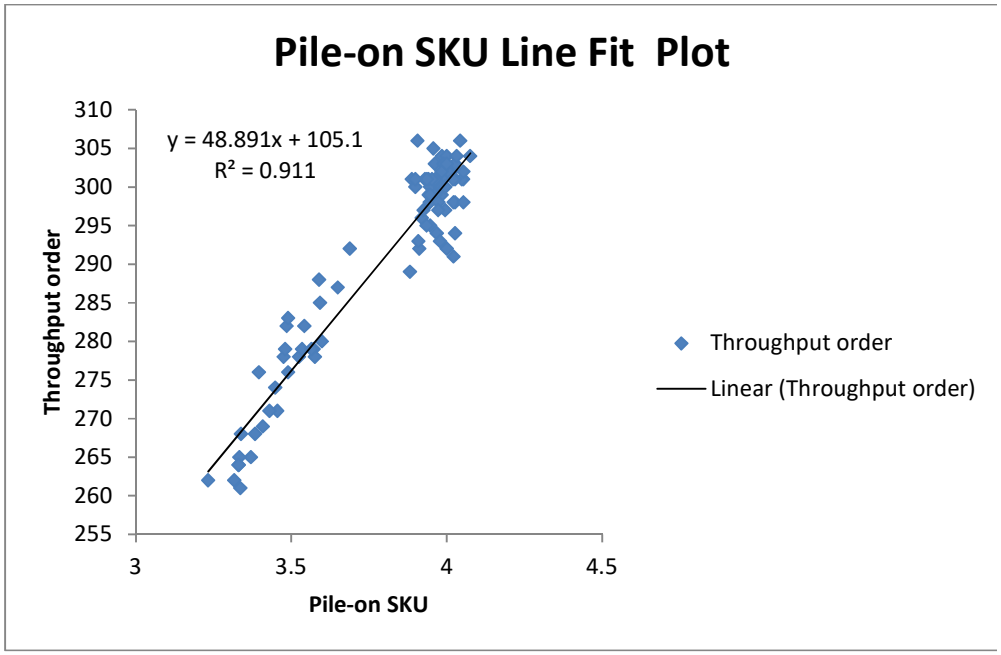

Figure 4-11 The fit plot of Throughput order (Y) and pile-on SKU (X)

Table 4-15 Regression statistics of Figure 4-11

Regression Statistics	
Multiple R	0.957322
R Square	0.916465
Adjusted R Square	0.915516
Standard Error	3.414197
Observations	90

Table 4-16 ANOVA test of Figure 4-11

	df	SS	MS	F	Significance F
Regression	1	11254	11254	965.4497	3.23E-49
Residual	88	1025.793	11.65674		
Total	89	12279.79			

Table 4-17 The significance of the coefficient and intercept of Figure 4-11

	Coefficients	Standard Error	t Stat	P-value	Lower 95%	Upper 95%	Lower 95.0%	Upper 95.0%
Intercept	125.8464	4.9361	25.4951	2.08 E-42	116.0369	135.6558	116.0369	135.6558
Pile-on SKU	104.5766	3.365655	31.07169	3.23 E-49	97.88805	111.2651	97.88805	111.2651

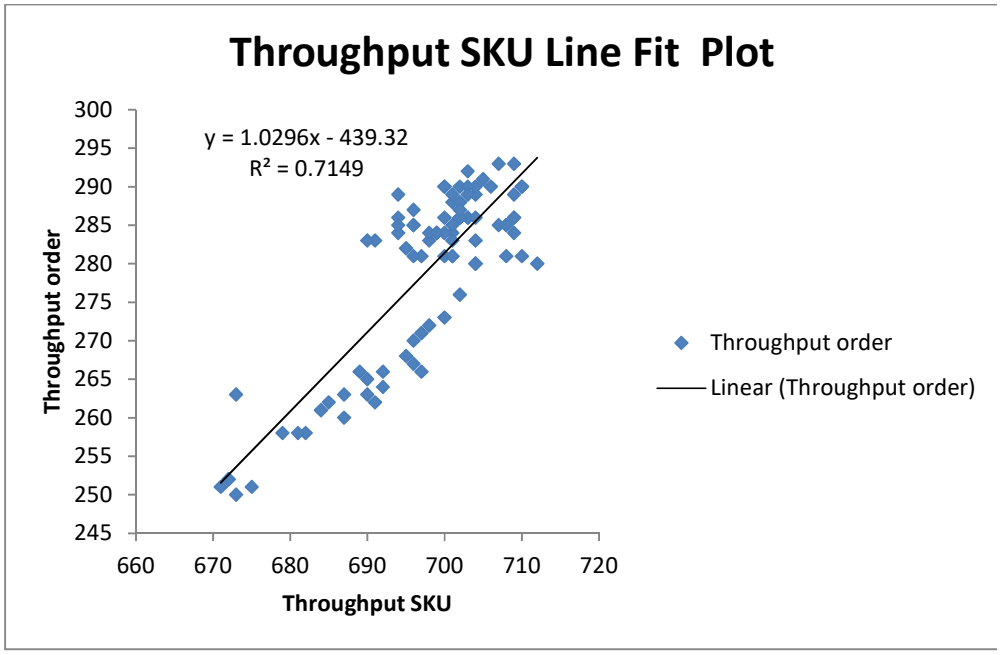

Figure 4-12 The fit plot of Throughput order (Y) and throughput SKU (X)

Table 4-18 Regression statistics of Figure 4-12

Regression Statistics	
Multiple R	0.845499
R Square	0.714869
Adjusted R Square	0.711629
Standard Error	6.307777
Observations	90

Table 4-19 ANOVA test of Figure 4-12

	df	SS	MS	F	Significance F
Regression	1	8778.441	8778.441	220.6301	1.05E-25
Residual	88	3501.348	39.78805		
Total	89	12279.79			

Table 4-20 The significance of the coefficient and intercept of Figure 4-12

	Coefficients	Standard Error	t Stat	P-value	Lower 95%	Upper 95%	Lower 95.0%	Upper 95.0%
Intercept	-439.316	48.35154	-9.08588	2.73E-14	-535.405	-343.228	-535.405	-343.228
Throughput SKU	1.029639	0.069319	14.8536	1.05E-25	0.891882	1.167396	0.891882	1.167396

CHAPTER 5
CONCLUSION AND FUTURE RESEARCH

This research focuses on optimizing the order picking process in the RMFS system. The two decision problems, POA and PPS, are solved simultaneously in the real-time decision scale. The proposed model maximizes the pile-on value every time the start condition of POA and PPS is triggered. That ensures every pod heading to the picking station has a high pile-on. The batching size of orders and pods in each decision is decided automatically and dynamically, depending on the current situation. Two objective functions are proposed to compare the effectiveness. One is purely maximizing the pile-on SKU. The other is maximizing the pile-on SKU with an additional term, representing the completion bonus if orders are being completed. The idea of the term is that the new order can be assigned and fulfilled once the order is completed. Finally, a heuristic method is proposed with the GRASP application to solve this NP-hard problem efficiently and effectively. Our proposed model is implemented in the simulation to emulate the environment of real industry. The result shows our proposed method performs significantly better in most indicators than the benchmark method.

This research conducts a regression analysis to compare the influence of different indicators on throughput order. The results show the order occupying bin time prevails in others. Therefore, we believe minimizing the order occupying bin time is one of the important issues to maximize the throughput order in the RMFS system, and we leave it to future research.

The proposed maximizing pile-on model is hard to solve. As a result, the performance of the proposed method is hard to examine in a large instance problem. We leave this to future work. More sensitive analysis are needed to test the robustness of the method. Expanding this research to a multiple-picking station problem is crucial to make it more applicable.

As mentioned in the first chapter, E-commerce orders usually have a very short delivery schedule. Furthermore, the orders may have different priorities to be completed depending on the different situations. Therefore, static, dynamic, or even hybrid methods are crucial to reply to the variant demand. There are many topics discussed in the traditional picker-to-parts warehouse. The same questions may also

be critical for the RMFS system at a strategic, tactical, or operational level. It is noted that the RMFS system is not only used in E-commerce warehouses. The characteristics of orders in different warehouses result in different solutions to most of the problems. For example, hypermarket orders usually have large quantities and lots of SKU in an order. Therefore, some assumptions in the literature no longer fit. In the end, extensive consideration of several decision problems is needed. For instance, how to minimize the AGV traveling distance given a sequence of pod visit? How do we integrate the picking and replenishment process? How do we cooperate with the AGV among many tasks in the RMFS?

REFERENCES

Absolunet. (2020). 10 eCommerce Trends 2020. https://10ecommercetrends.com/

Aldarondo, F. J., & Bozer, Y. A. (2022). Expected distances and alternative design configurations for automated guided vehicle-based order picking systems. *International Journal of Production Research*, *60*(4), 1298-1315.

Bertsimas, D., & Tsitsiklis, J. (1993). Simulated annealing. *Statistical science*, *8*(1), 10-15.

Blum, C., & Roli, A. (2003). Metaheuristics in combinatorial optimization: Overview and conceptual comparison. *ACM computing surveys (CSUR)*, *35*(3), 268-308.

Boysen, N., Briskorn, D., & Emde, S. (2017). Parts-to-picker based order processing in a rack-moving mobile robots environment. *European Journal of Operational Research*, *262*(2), 550-562.

Chen, M.-C., & Wu, H.-P. (2005). An association-based clustering approach to order batching considering customer demand patterns. *Omega*, *33*(4), 333-343.

Cheng, C.-Y., Chen, Y.-Y., Chen, T.-L., & Yoo, J. J.-W. (2015). Using a hybrid approach based on the particle swarm optimization and ant colony optimization to solve a joint order batching and picker routing problem. *International Journal of Production Economics*, *170*, 805-814.

Clarke, G., & Wright, J. W. (1964). Scheduling of vehicles from a central depot to a number of delivery points. *Operations research*, *12*(4), 568-581.

Dallari, F., Marchet, G., & Melacini, M. (2009). Design of order picking system. *The international journal of advanced manufacturing technology*, *42*(1), 1-12.

De Koster, R., Le-Duc, T., & Roodbergen, K. J. (2007). Design and control of warehouse order picking: A literature review. *European Journal of Operational Research*, *182*(2), 481-501.

Eglese, R. W. (1990). Simulated annealing: a tool for operational research. *European Journal of Operational Research*, *46*(3), 271-281.

Feo, T. A., & Resende, M. G. (1995). Greedy randomized adaptive search procedures. *Journal of global optimization*, *6*(2), 109-133.

Graham, R. L. (1966). Bounds for certain multiprocessing anomalies. *Bell system technical journal*, *45*(9), 1563-1581.

Hansen, P., Mladenović, N., Brimberg, J., & Pérez, J. A. M. (2019). Variable neighborhood search. In *Handbook of metaheuristics* (pp. 57-97). Springer.

Henderson, D., Jacobson, S. H., & Johnson, A. W. (2003). The theory and practice of simulated annealing. In *Handbook of metaheuristics* (pp. 287-319). Springer.

Ho, Y.-C., & Tseng, Y.-Y. (2006). A study on order-batching methods of order-picking in a distribution centre with two cross-aisles. *International Journal of Production Research*, *44*(17), 3391-3417.

Introduction of Beam Search. https://en.wikipedia.org/wiki/Beam_search#cite_note-2

Jones, T. (1995a). *Evolutionary algorithms, fitness landscapes and search* Citeseer].

Jones, T. (1995b). One operator, one landscape. *Santa Fe Institute Technical Report*, 95-02.

Lamballais, T., Merschformann, M., Roy, D., de Koster, M., Azadeh, K., & Suhl, L. (2022). Dynamic policies for resource reallocation in a robotic mobile fulfillment system with time-varying demand. *European journal of operational research*, *300*(3), 937-952.

Lamballais, T., Roy, D., & De Koster, M. (2017). Estimating performance in a robotic mobile fulfillment system. *European Journal of Operational Research*, *256*(3), 976-990.

Li, X., Hua, G., Huang, A., Sheu, J.-B., Cheng, T., & Huang, F. (2020). Storage assignment policy with awareness of energy consumption in the Kiva mobile fulfilment system. *Transportation Research Part E: Logistics and Transportation Review*, *144*, 102158.

Merschformann, M., Lamballais, T., De Koster, M., & Suhl, L. (2019). Decision rules for robotic mobile fulfillment systems. *Operations Research Perspectives*, *6*, 100128.

Merschformann, M., Lamballais, T., & De Koster, R. (2018). Decision Rules for Robotic Mobile Fulfillment Systems. *Operations Research Perspectives*, *6*. https://doi.org/10.1016/j.orp.2019.100128

Merschformann, M., Xie, L., & Li, H. (2017). RAWSim-O: A simulation framework for robotic mobile fulfillment systems. *arXiv preprint arXiv:1710.04726*.

Mitchell, M. (1998). *An introduction to genetic algorithms*. MIT press.

MWPVL. (2012). *Leadership in Global Supply Chain and Logistics Consulting*. https://www.mwpvl.com/html/kiva_systems.html

Pitsoulis, L. S., & Resende, M. G. (2002). Greedy randomized adaptive search procedures. *Handbook of applied optimization*, 168-183.

Shi, X., Deng, F., Fan, Y., Ma, L., Wang, Y., & Chen, J. (2021). A Two-Stage Hybrid Heuristic Algorithm for Simultaneous Order and Rack Assignment Problems. *IEEE Transactions on Automation Science and Engineering*.

Statista. (2020). *eCommerce worldwide*. https://www.statista.com/outlook/dmo/ecommerce/worldwide

Valle, C. A., & Beasley, J. E. (2021). Order allocation, rack allocation and rack sequencing for pickers in a mobile rack environment. *Computers & Operations Research*, *125*, 105090.

Wulfraat, M. (2012). *Is Kiva systems a good fit for your distribution center? An unbiased distribution consultant evaluation.* https://mwpvl.com/html/kiva_systems.html

Xie, L., Thieme, N., Krenzler, R., & Li, H. (2021). Introducing split orders and optimizing operational policies in robotic mobile fulfillment systems. *European Journal of Operational Research*, *288*(1), 80-97.

Yuan, R., Graves, S. C., & Cezik, T. (2019). Velocity-Based Storage Assignment in Semi-Automated Storage Systems. *Production and Operations Management*, *28*(2), 354-373.

Zhou, M., Ding, Z., Tang, J., & Yin, D. (2018). Micro behaviors: A new perspective in e-commerce recommender systems. Proceedings of the eleventh ACM international conference on web search and data mining,

Zhuang, Y., Zhou, Y., Yuan, Y., Hu, X., & Hassini, E. (2021). Order picking optimization with rack-moving mobile robots and multiple workstations. *European Journal of Operational Research*.

Zou, B., Gong, Y., Xu, X., & Yuan, Z. (2017). Assignment rules in robotic mobile fulfilment systems for online retailers. *International Journal of Production Research*, *55*(20), 6175-6192.